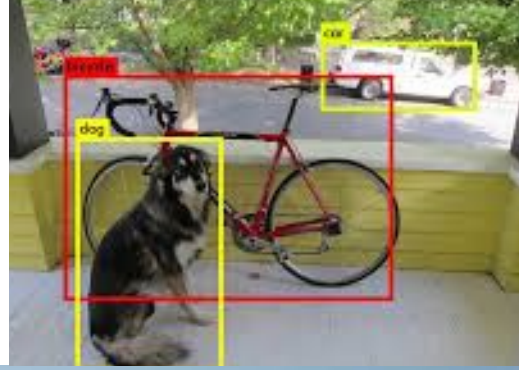
Deep Learning in Computer Vision

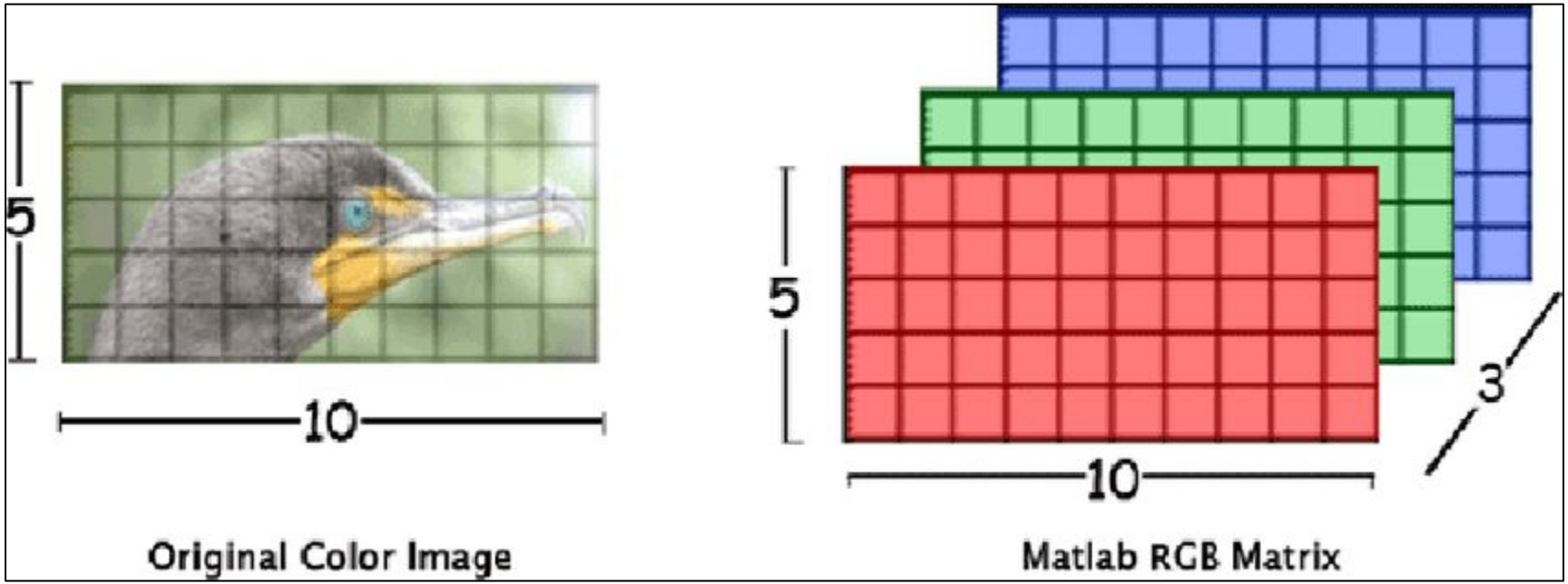
Kushal Vyas
kushalvyas.github.io

So, What is Computer Vision

- High-level understanding of Images and Videos
- E.g include :
  - Image Enhancement
  - Image Recognition
  - Feature Extraction
  - Stereo Vision
  - Pose Estimation
  - Registration, Stitching
  - … many more …
- Best example of a real time Vision system is a self-driving car



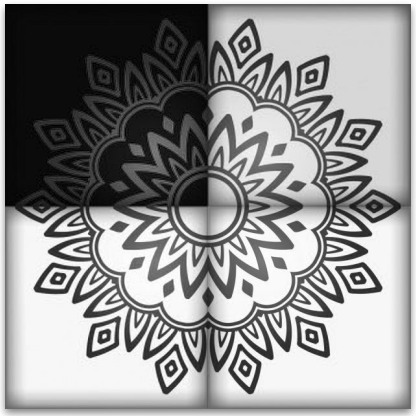
Images & Pixels



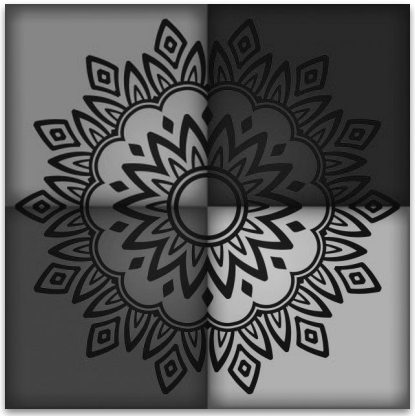
Representing Color spaces

- Consider an Image : 32 x 32 x 3
- R, G, B Channel image.
- When you split each channel , you get

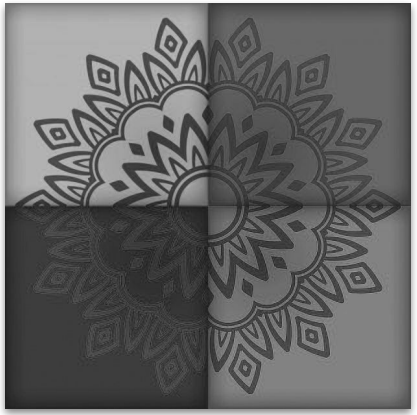
ORIGINAL



RED CHANNEL



GREEN CHANNEL



BLUE CHANNEL

Darker shade represents a value approaching 0 ; Whiter shade represents a value approaching 255

Tools for CV

Pure Computer Vision

- OpenCV
- DLib
- ImageJ
- SimpleCV
- VLFeat
- VxL
- Tracking.JS
- FastCV
- BoofCV

Deep Learning with Vision

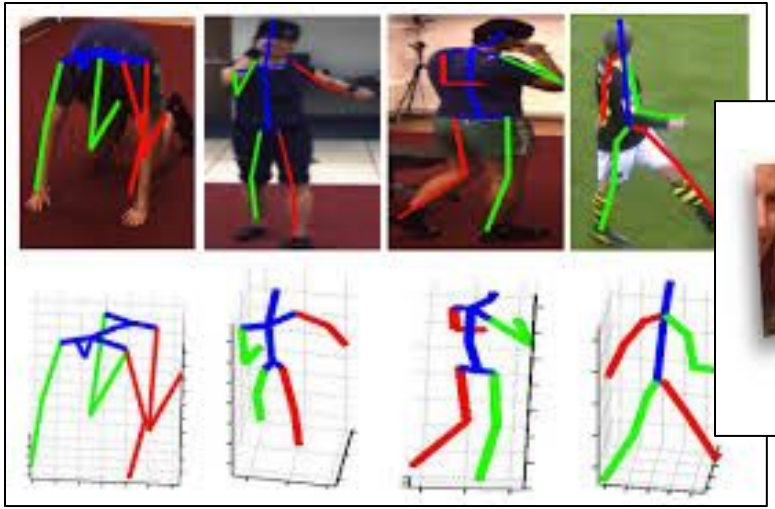
- Tensorflow
- Caffe
- Keras
- Theano
- Also, use scipy, skimage or PIL for preprocessing

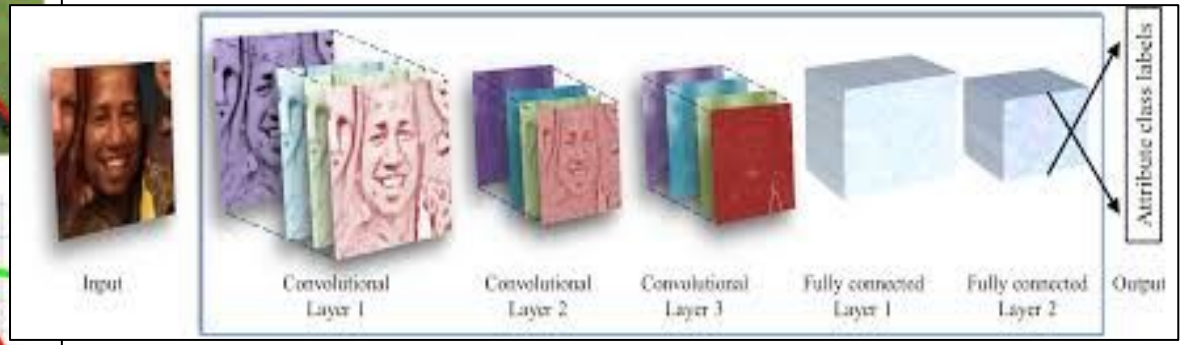
- Python is one of the preferred languages in either areas.

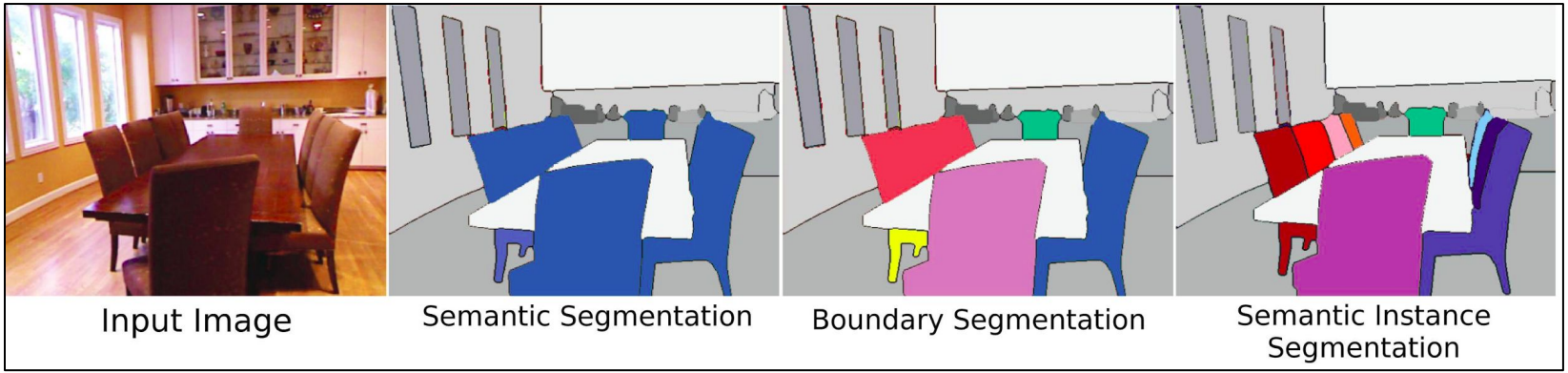
Current Trends

- Object Classification
- Object Detection
- Artistic Style Transfer
- Semantic Segmentation
- Matting
- Pose Estimation
- 3D Segmentation
- Generation of Primitives
- GANS
- 3D matching of local descriptors

Other Examples

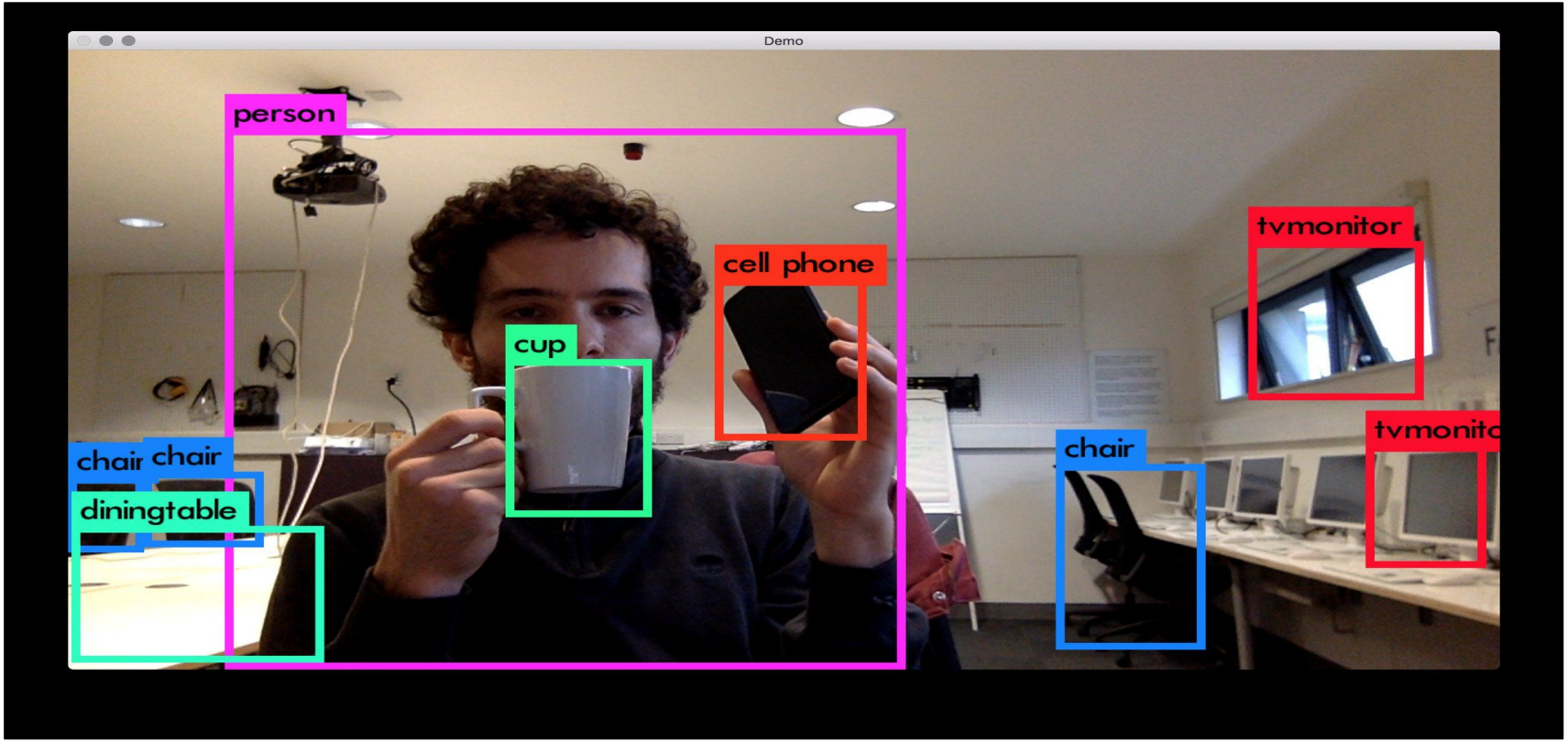






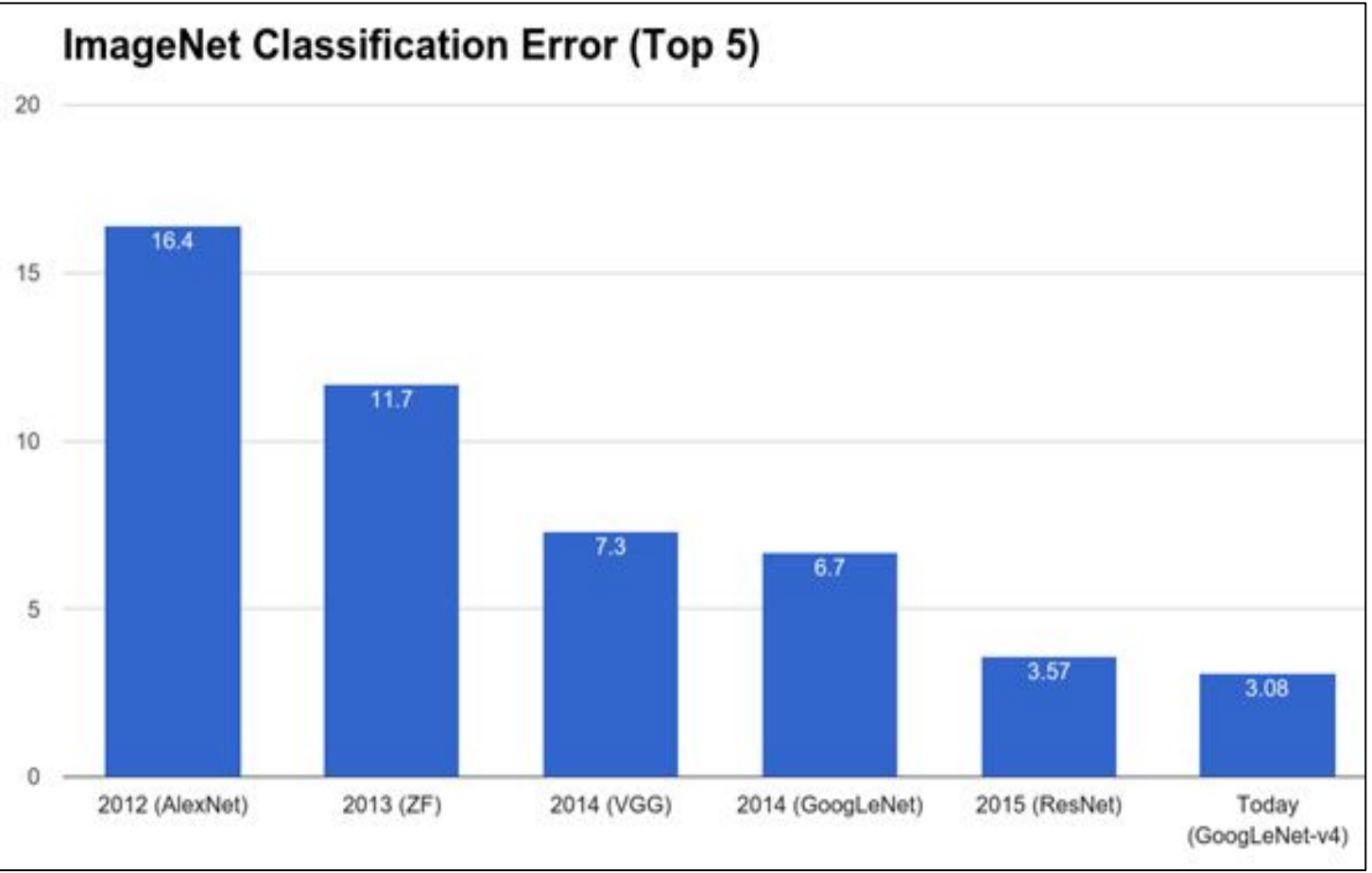
Object Classification

- The task of correctly recognizing the image and its label.
- Stanford ImageNET Large Scale Visual Recognition Challenge
- IMAGENET : Amazing CNN’s have come out of them
- 22K categories and 14M images



Object Recognition and Detection using YOLO architecture

Top Stats from ImageNET

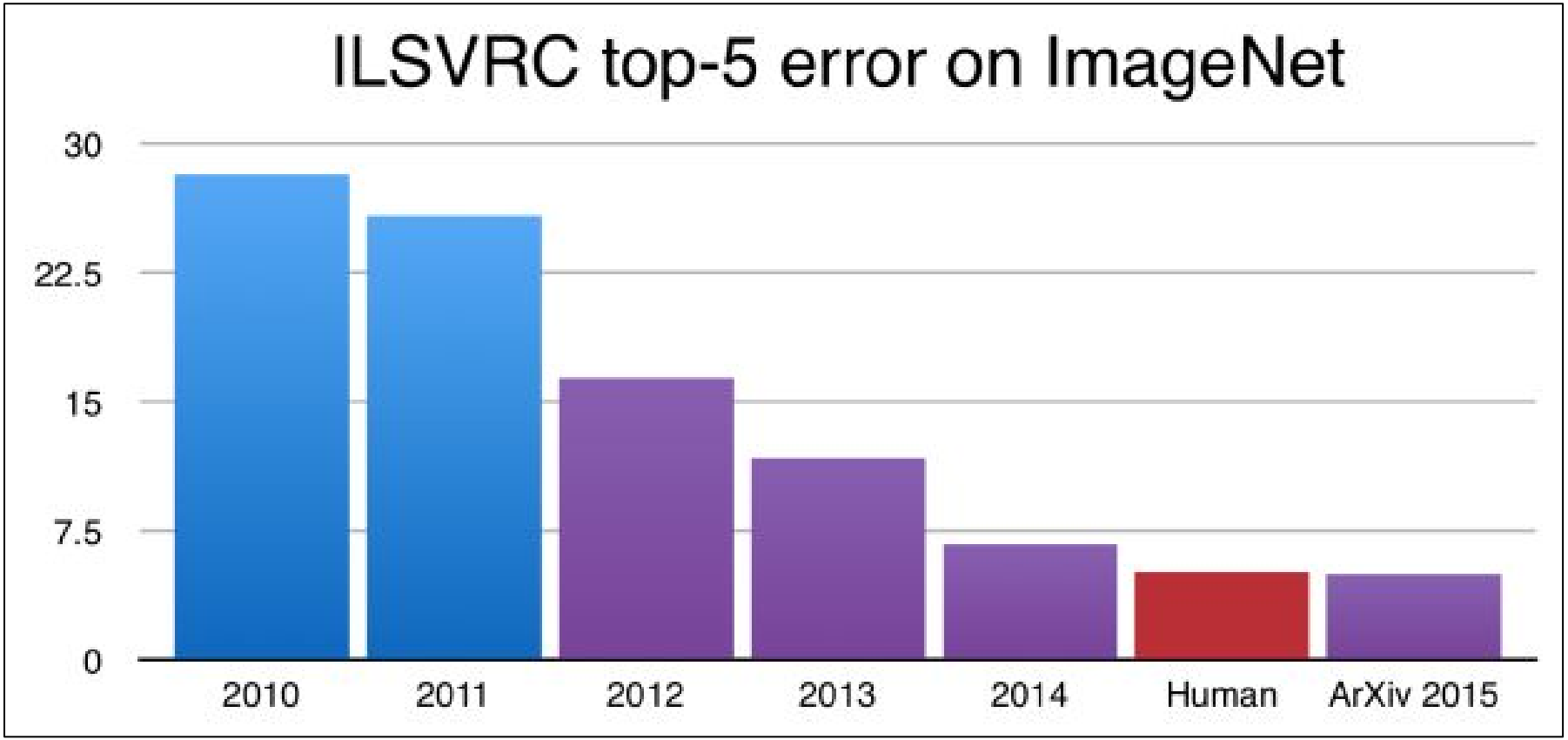


#FACT

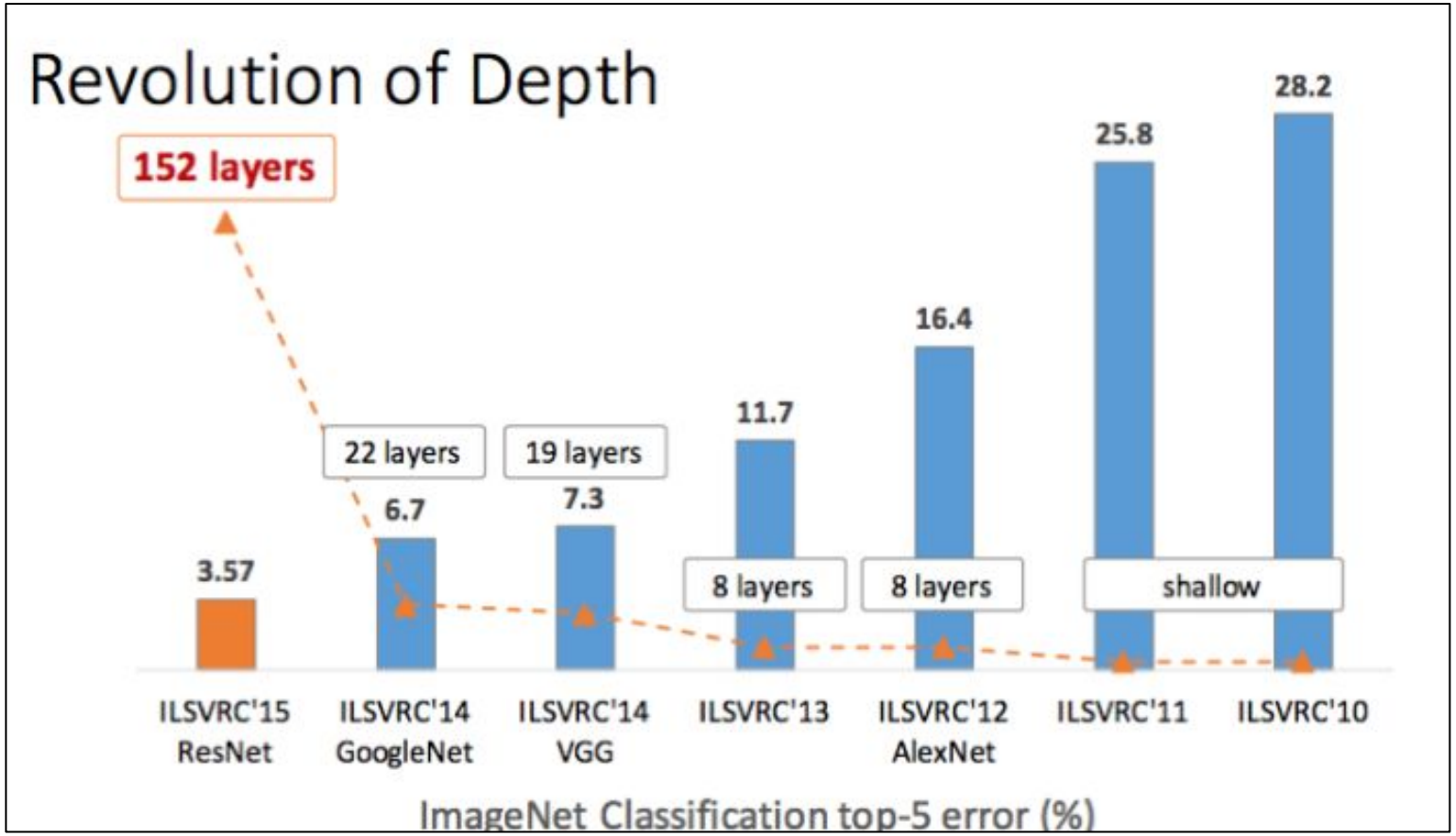
Human level error for ImageNET is approx 5%.

Today's CNN has beaten human at the task of image classification

Comparing with humans …

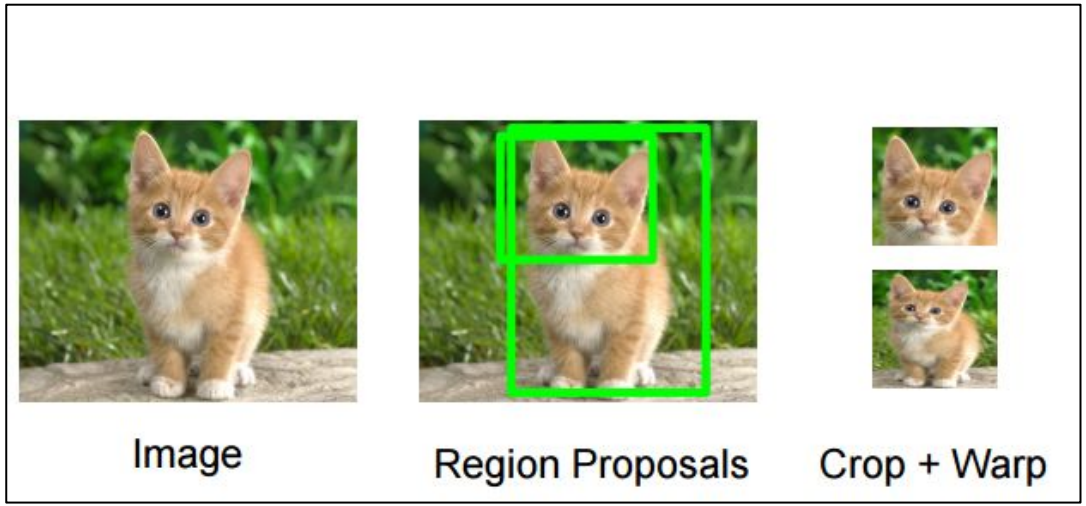


Layer-wise comparison



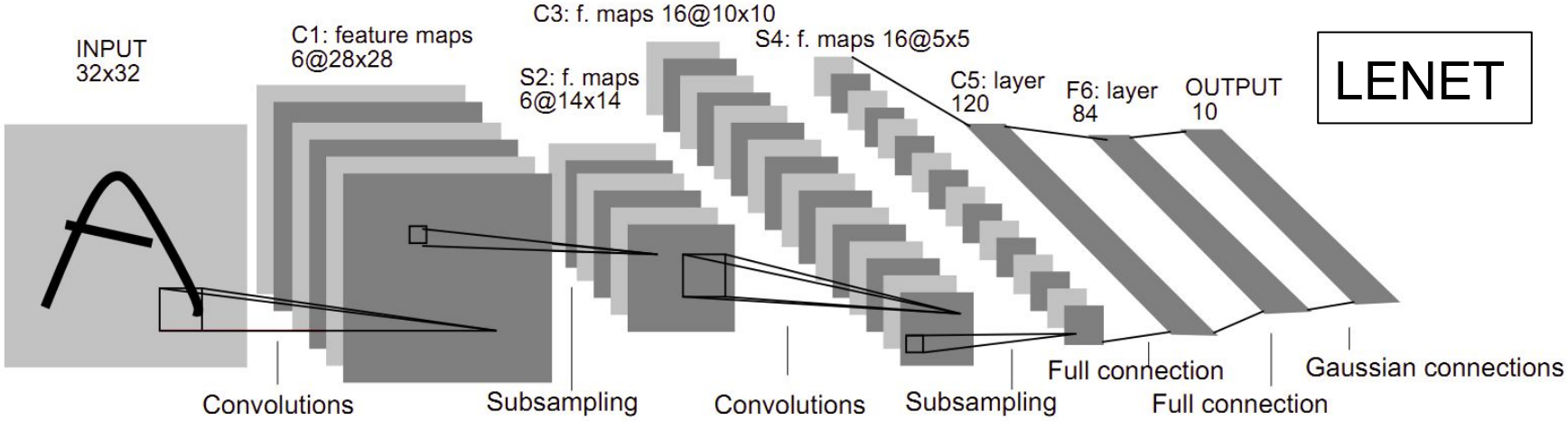
Localization of Objects : The second part of classification

- After classification, the next interesting problem in ImageNET
- To locate and predict a bounding box around the classified object

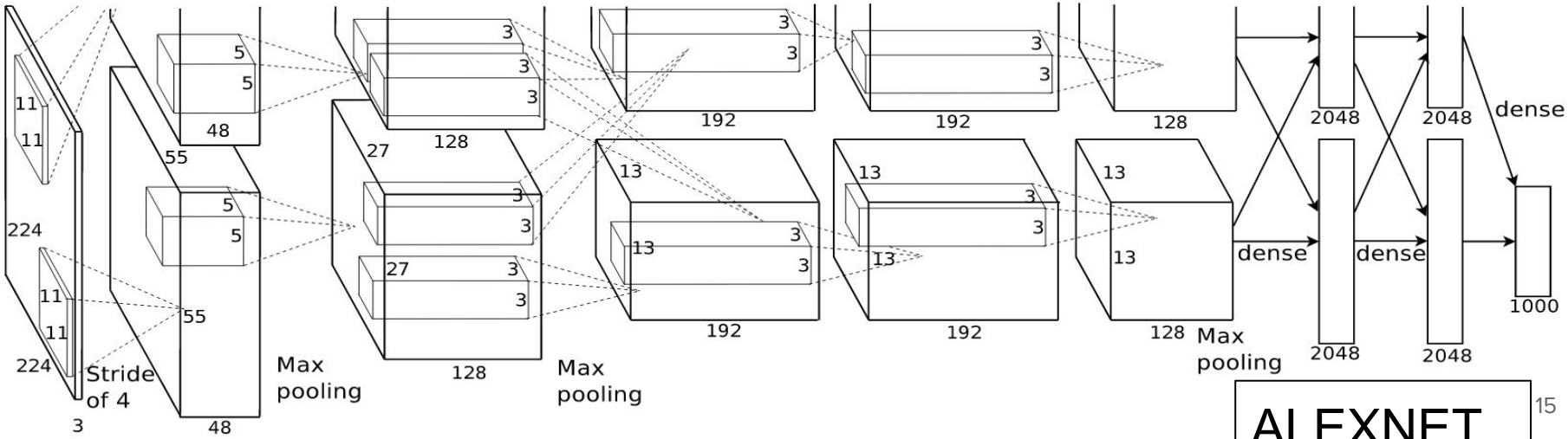


ALEXNET : Winner ILSVRC 2012
LeNET : Yann LeCun et. al. , 1989

- AlexNET : Error of ~ 16 % in ILSVRC
- Developed by
  - Alex Krizhevsky
  - Ilya Sutskever
  - Geoff Hinton (AI's GODFATHER) (Recently published CapsNET)
  - Basically, the full Deep Learning community is going crazy behind CapsNET
- Not the first implementation of CNNS; Yann LeCun et. al. LENET

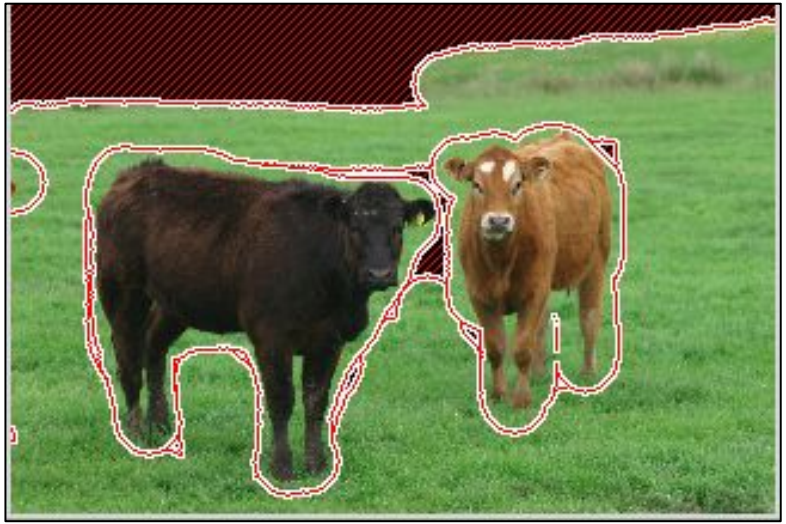


LENET



ALEXNET

Semantic Segmentation

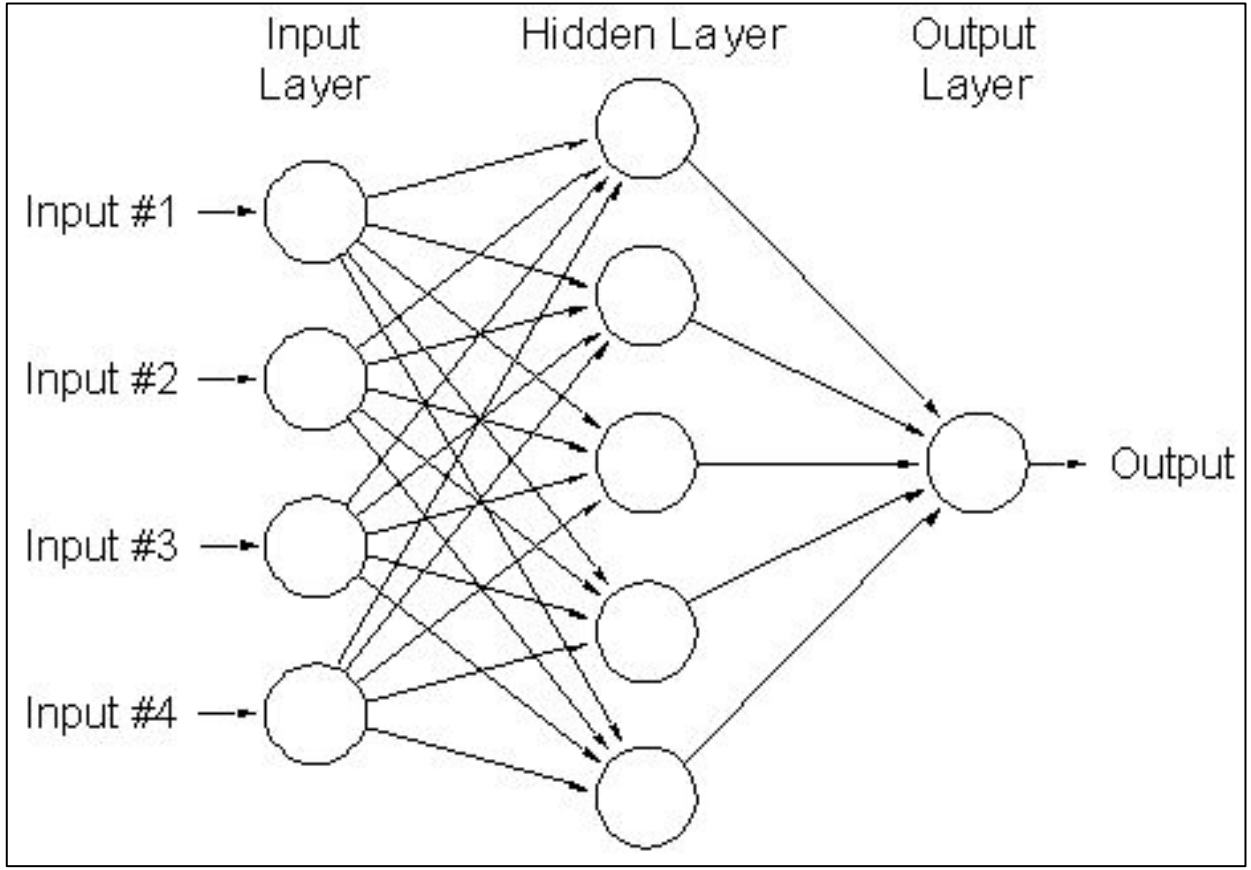


Extract Exact Image Portions

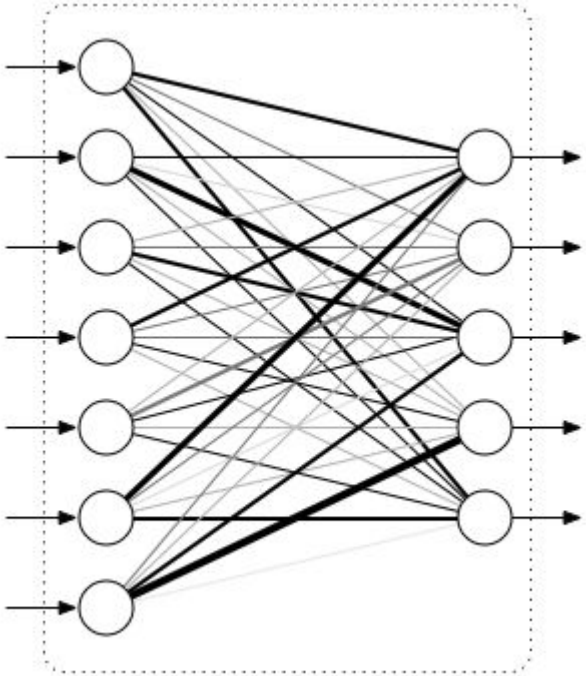
Capturing Instance Level Information

Normal Neural Networks

- Contains vectorized Inputs
- Hidden Layers
- Binary/ Probabilistic Outputs
- Training follows delta rule

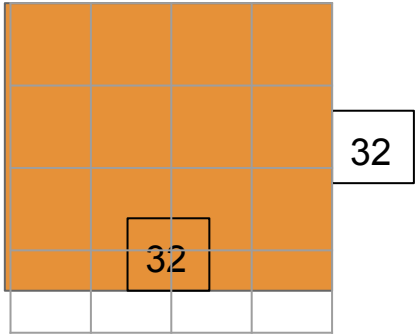


Fully Connected Layers

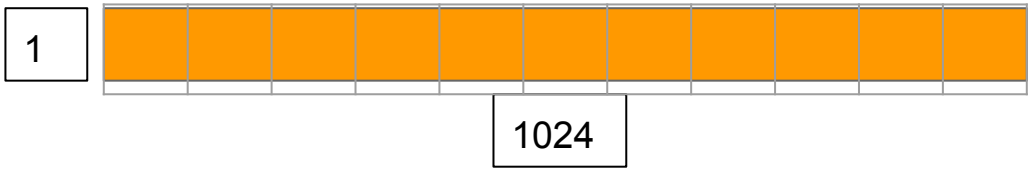


All neurons are connected to each other in a FC layer

- Example : Consider a 32 x 32 matrix.
- Represented as :

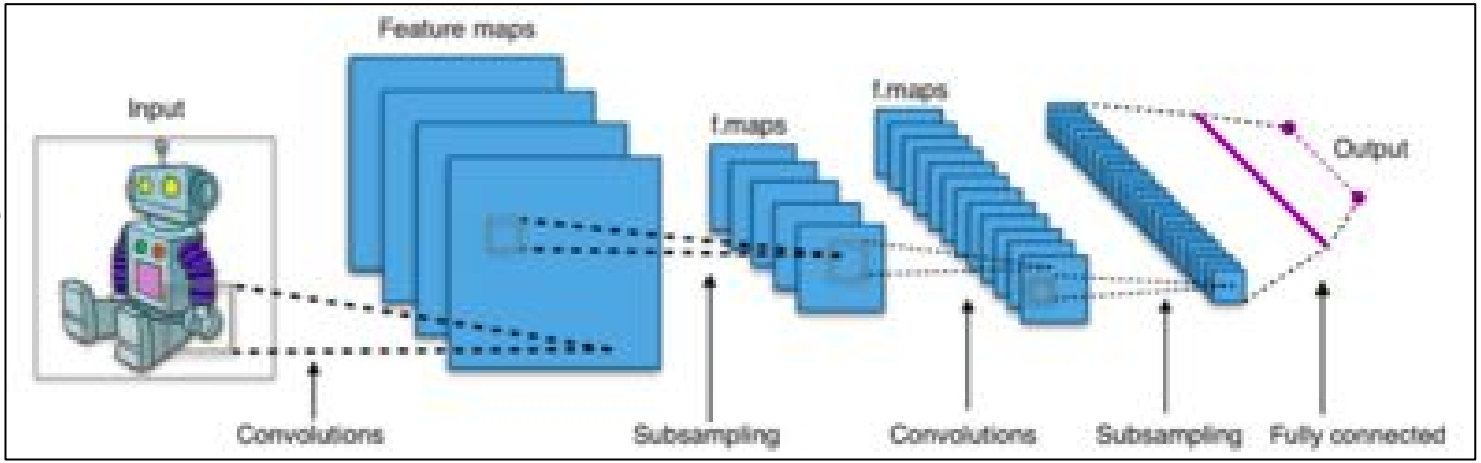


- Fully Connected / Dense Layer flattens to 1D Vector



Convolutional Neural Networks

- Input:
  2D Image



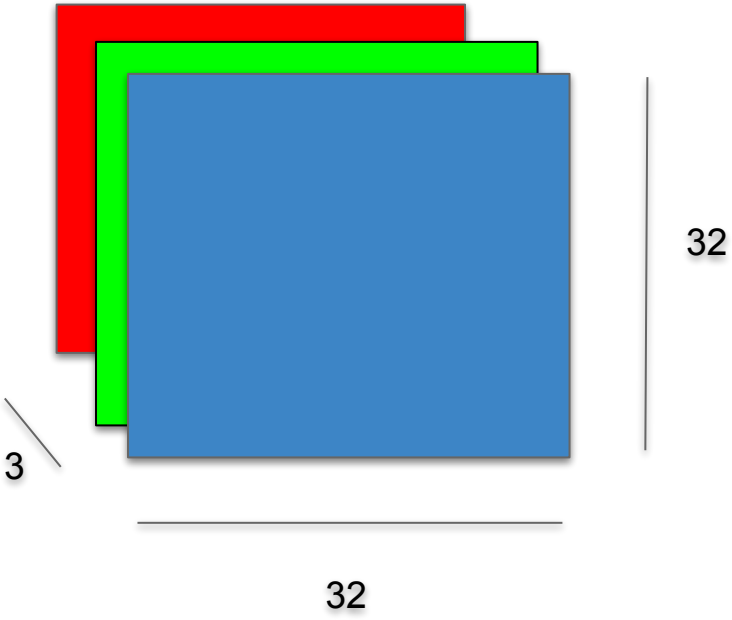
- Computes multiple convolutions throughout . [Convolutional Layers]
- Subsamples output. Repeats convolutions
- Ends with a Fully Connected Layers
- Output probability of classes

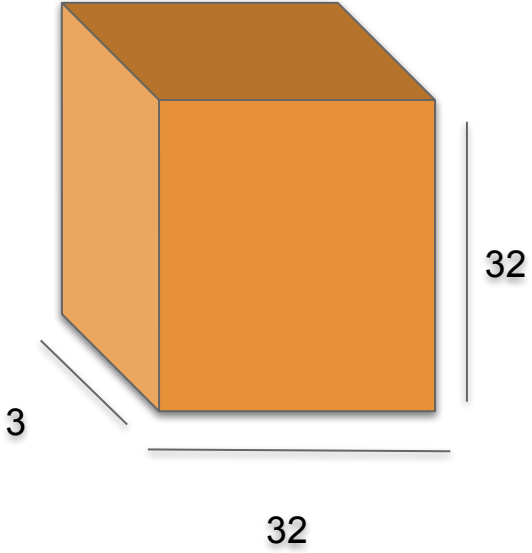
Architecture of a Simple ConvNet

- Tensor : Let tensor be a matrix (singular or multi-ranked & multidimensional)
- Conv : A convolutional layer
- Activation Layer : activation of signal produced by neuron of neural network
- Pooling layer : Downsampling.
- FC layer / Dense Layer: eventual separator - classifier neural net

Convolution Layers

- Consider an Image: 32 x 32 x 3





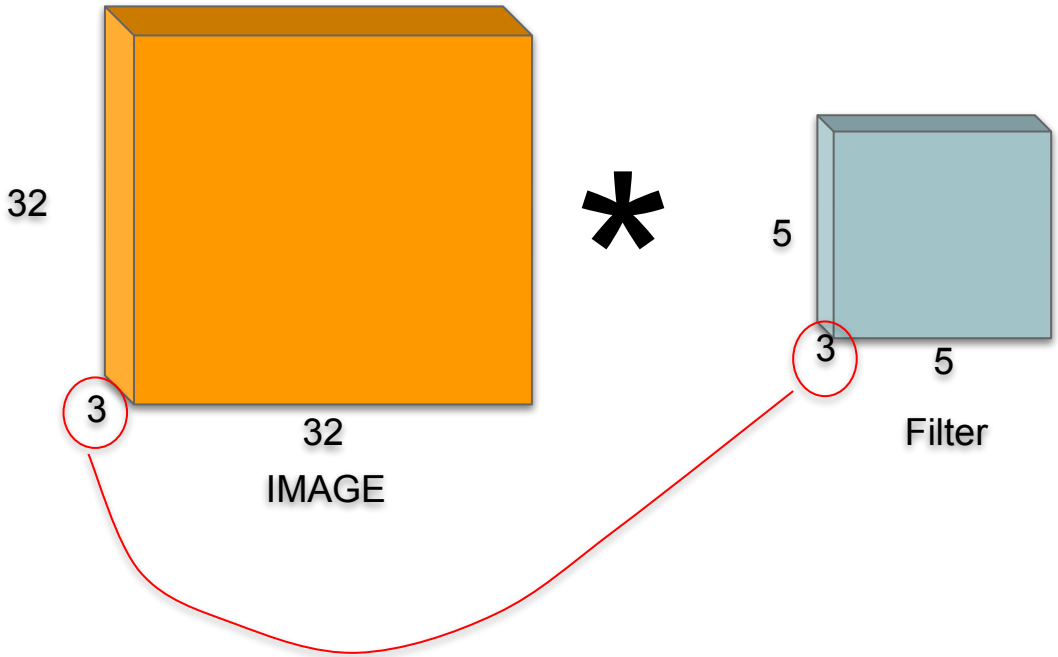


Image : 32 x 32 x 3
Filter for convolution : 5 x 5 x 3

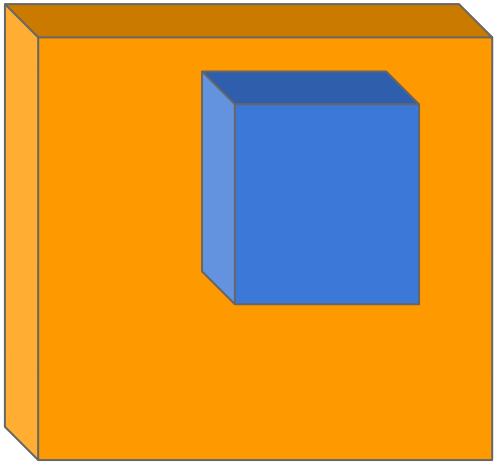
Note: The depth dimension of the input image to the CNN is equal to the depth

The filter is slided over the image, pixel by pixel.

Since the filter depth is same as the image depth, there is complete depth coverage in the convolution layer

Convolving the image and the kernel

Convolutional Layers .... contd.

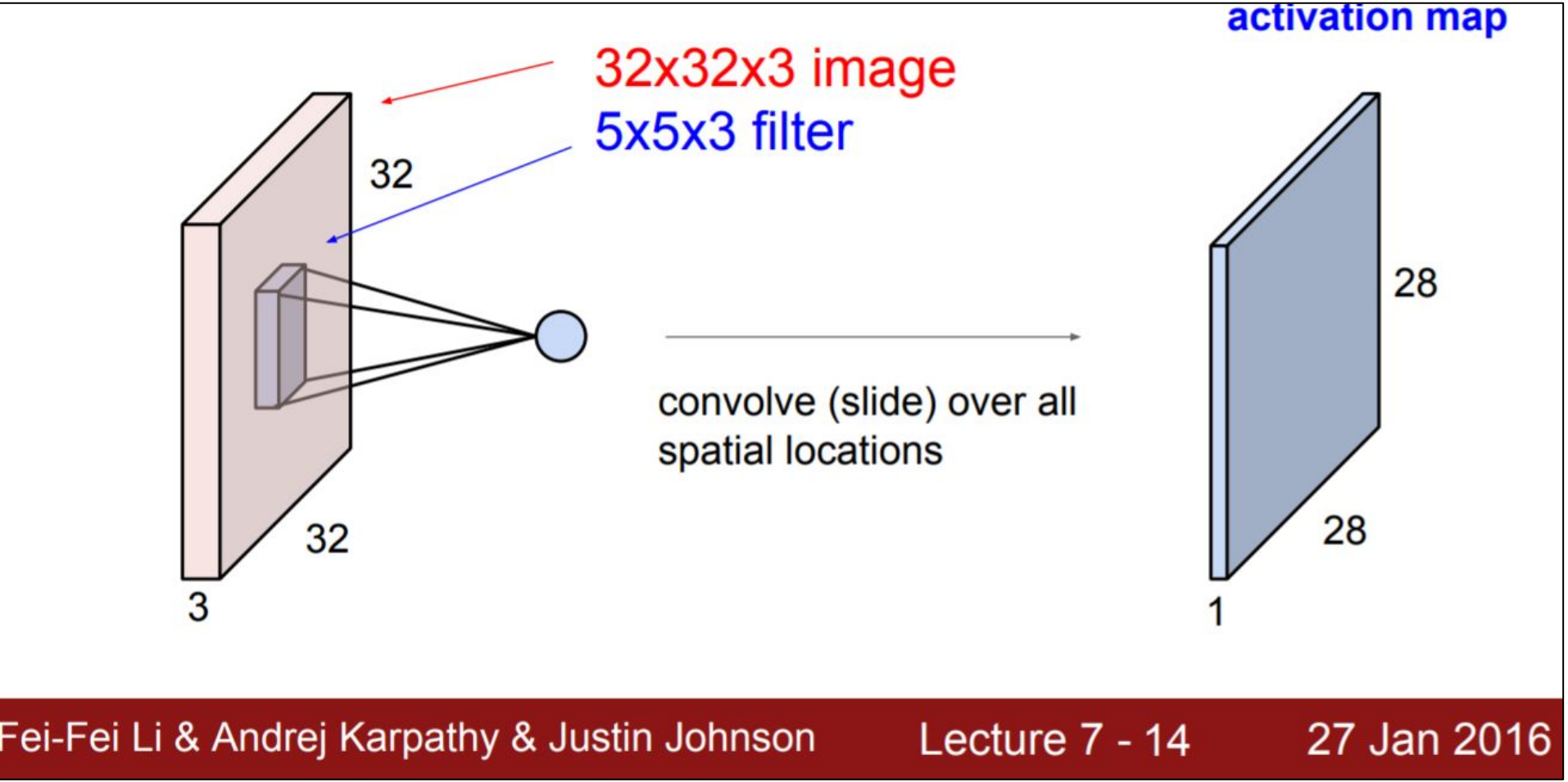


Slide (convolve) filter over all possible spatial locations

IMAGE: 32 X 32 X 3
FILTER: 5 X 5 X 3

CONVOLVED OUTPUT
28 X 28 X 1

Since this is a complete depth-through operation, the output has only 1 - dimension.
However, the stacked output will be equal to number of filters.



Credits : CS231N , Stanford

Conv Layer - Math

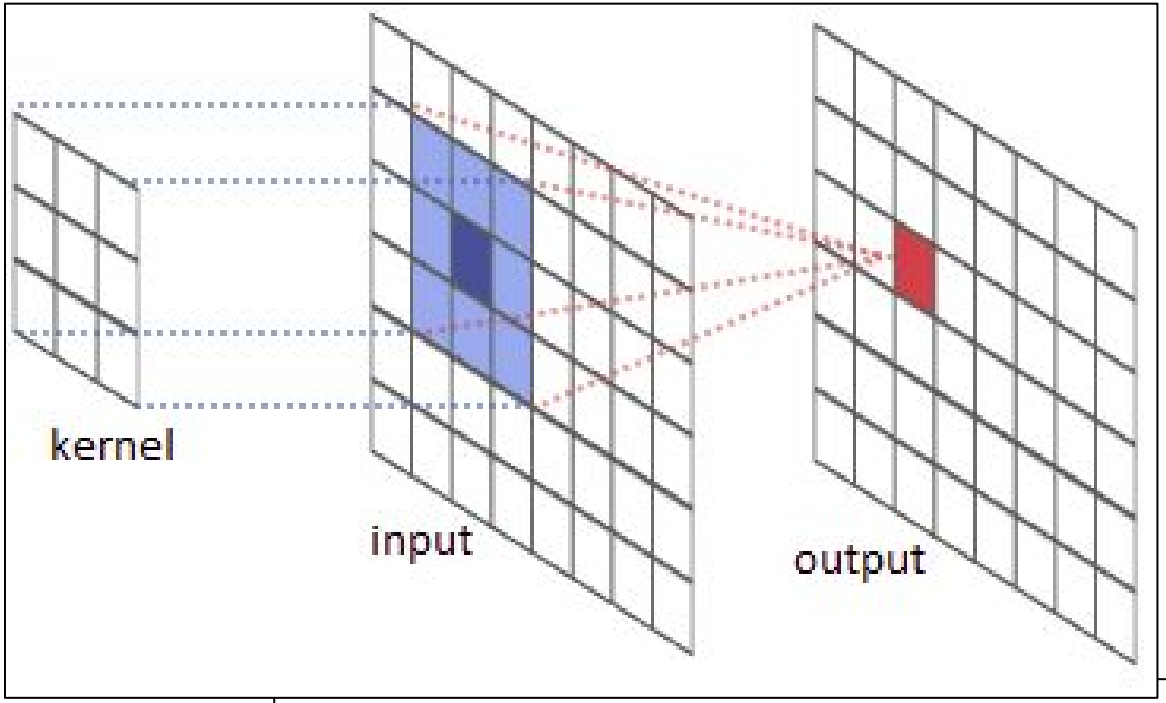
- Its a dot product across all spatial locations
- Dot is summed through the depth dimension

- Which is why
- 32 x 32 x 3 image , convolved with 5 x 5x 3 filter outputs
- 28 x 28 x 1

- Basically, this is convolution :

$$y[n] = x[n] ** h[n] = \sum_{k=-\infty}^{+\infty} x[k]h[n-k]$$

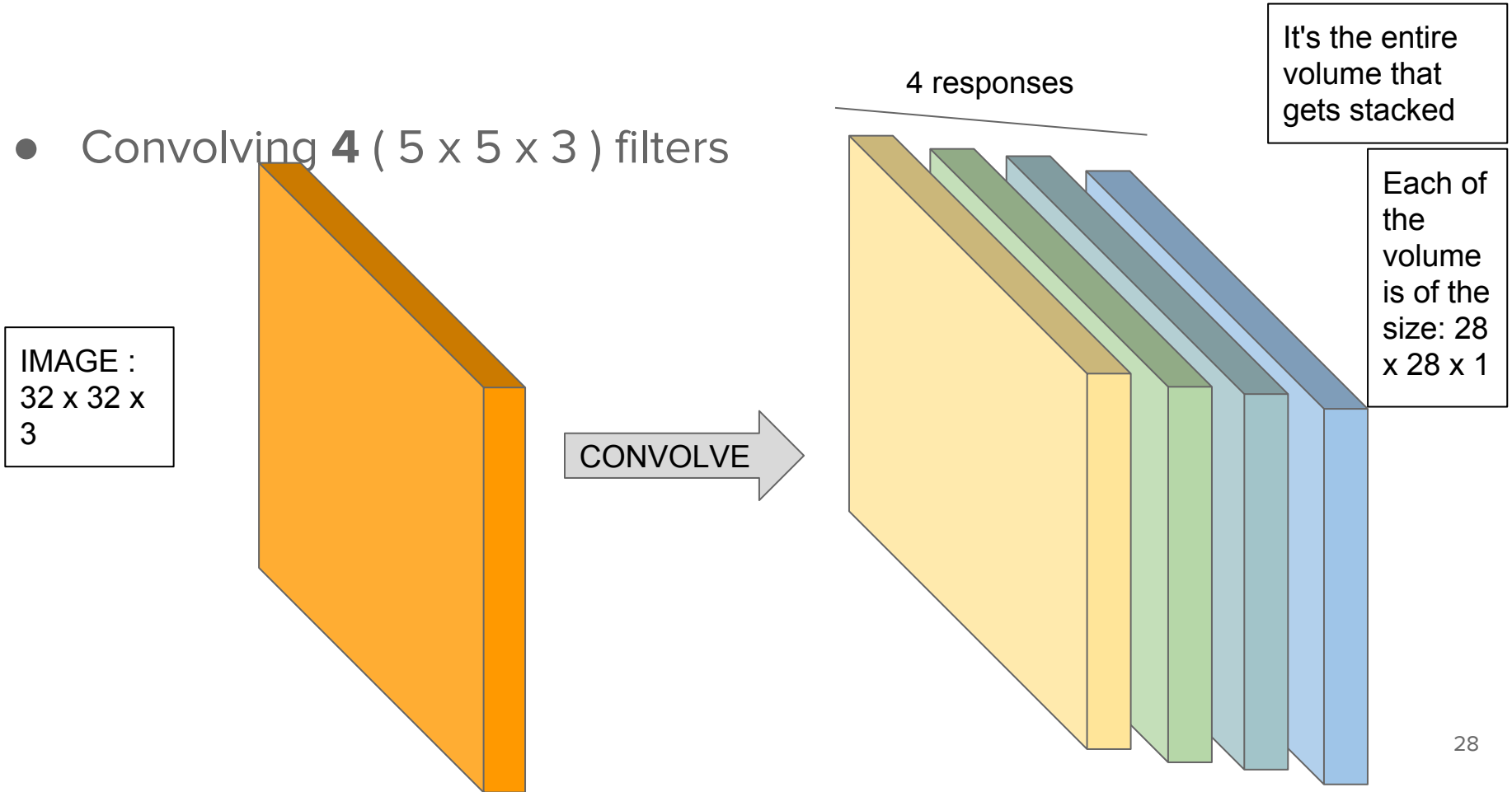
- But CNN has more of a correlation & dot product stuff...



the result of taking a dot product between the filter and a small 5x5x3 chunk of the image

Increasing Filter Banks

- Convolving **4** (5 x 5 x 3) filters



How spatial convolutions occur
Consider 7 x 7 Matrix: With 3 x 3 filter

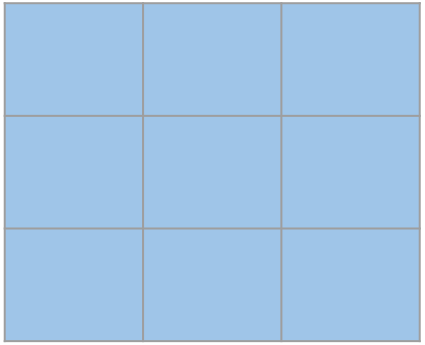
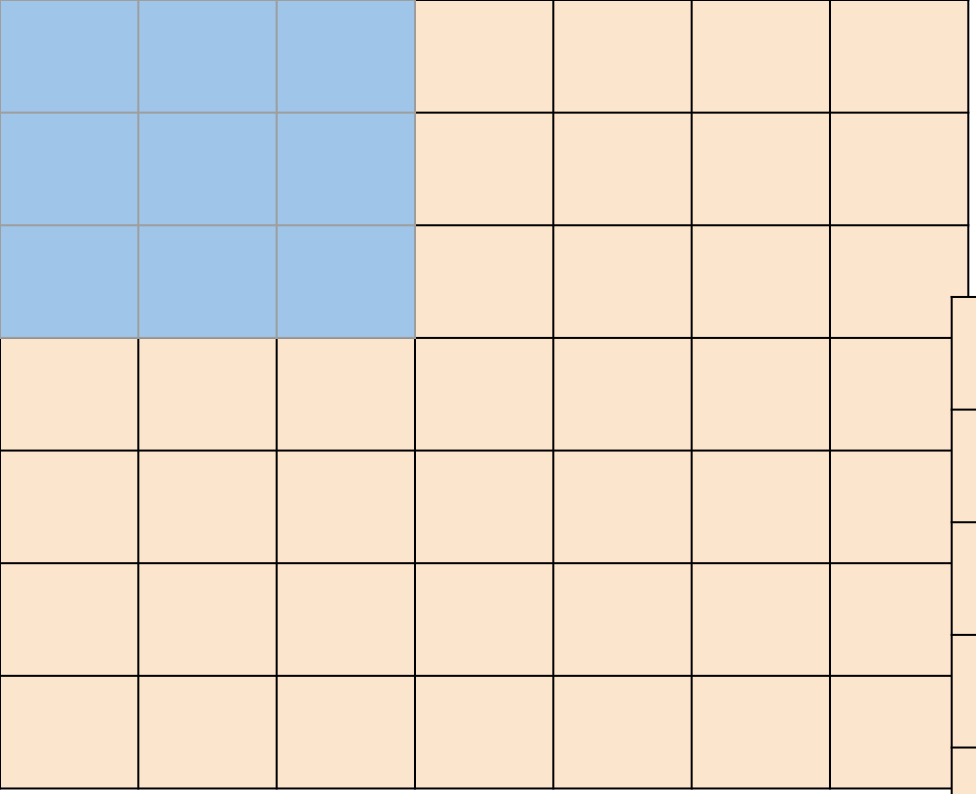
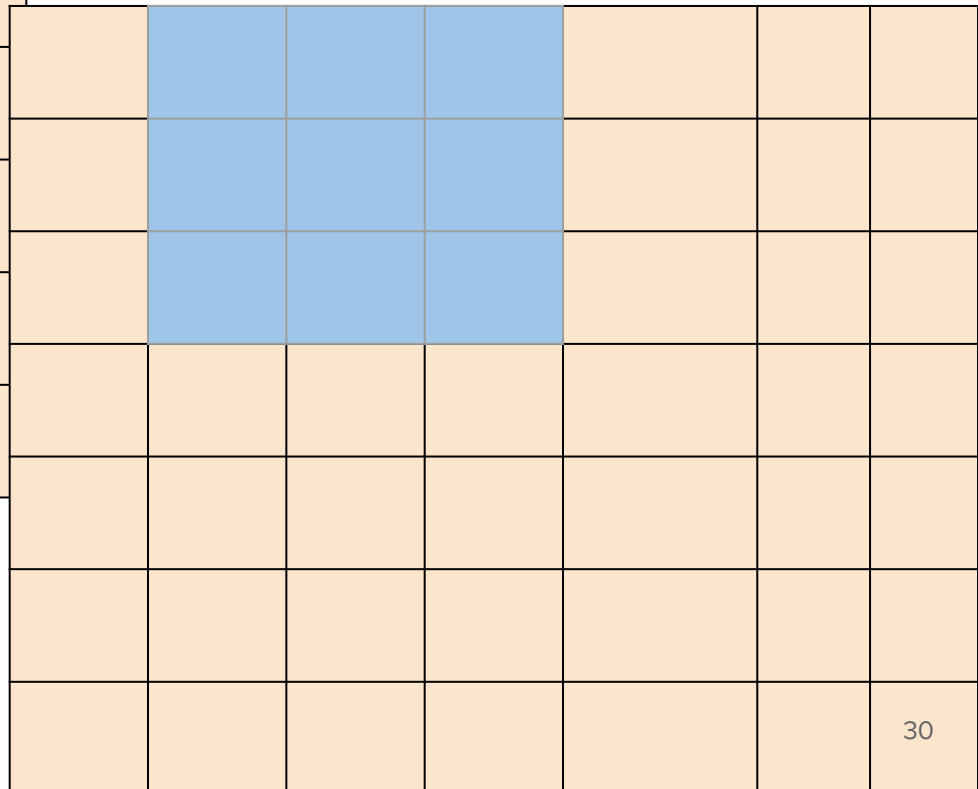


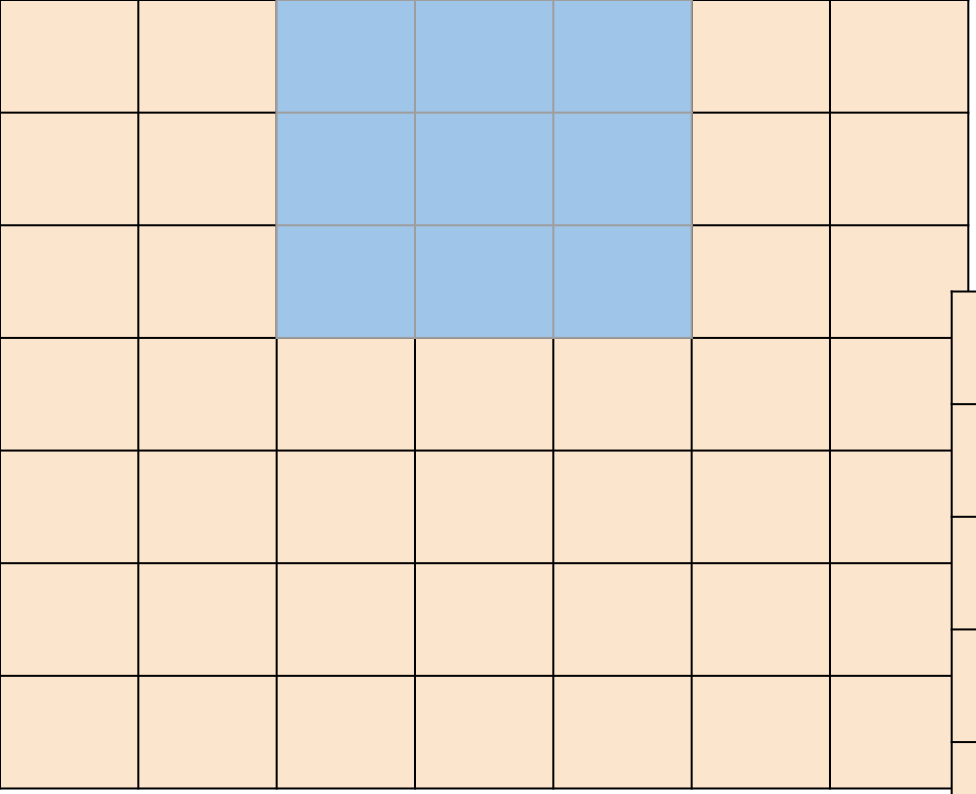
Image : 32 x 32
Filter: 3 x 3



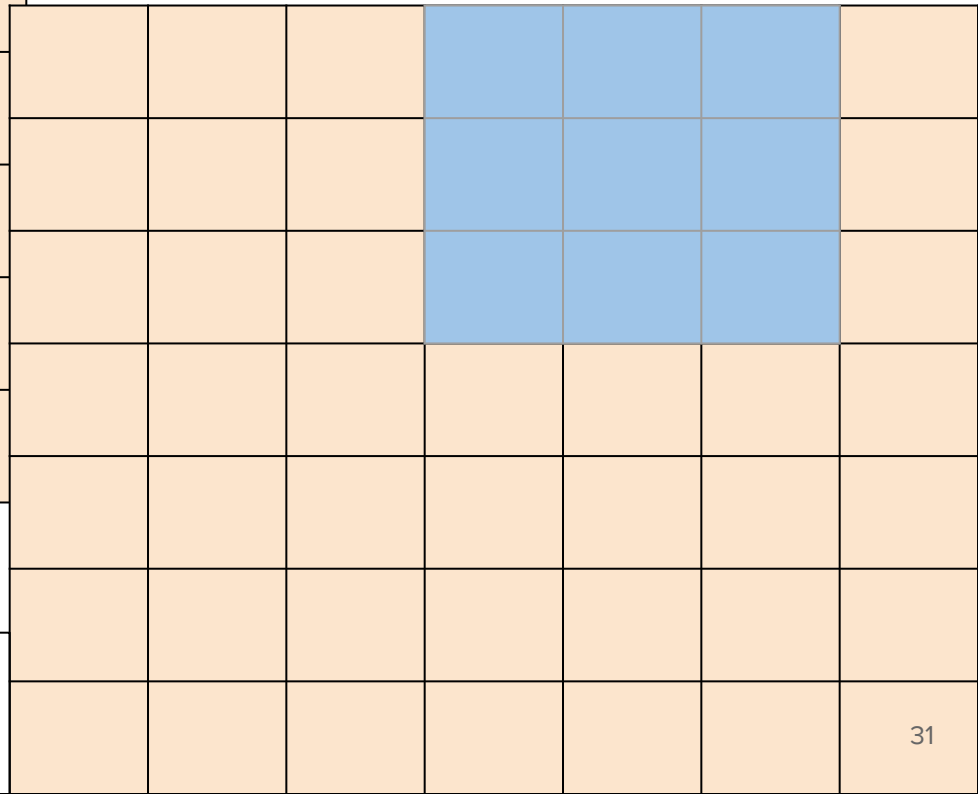
Slide 1



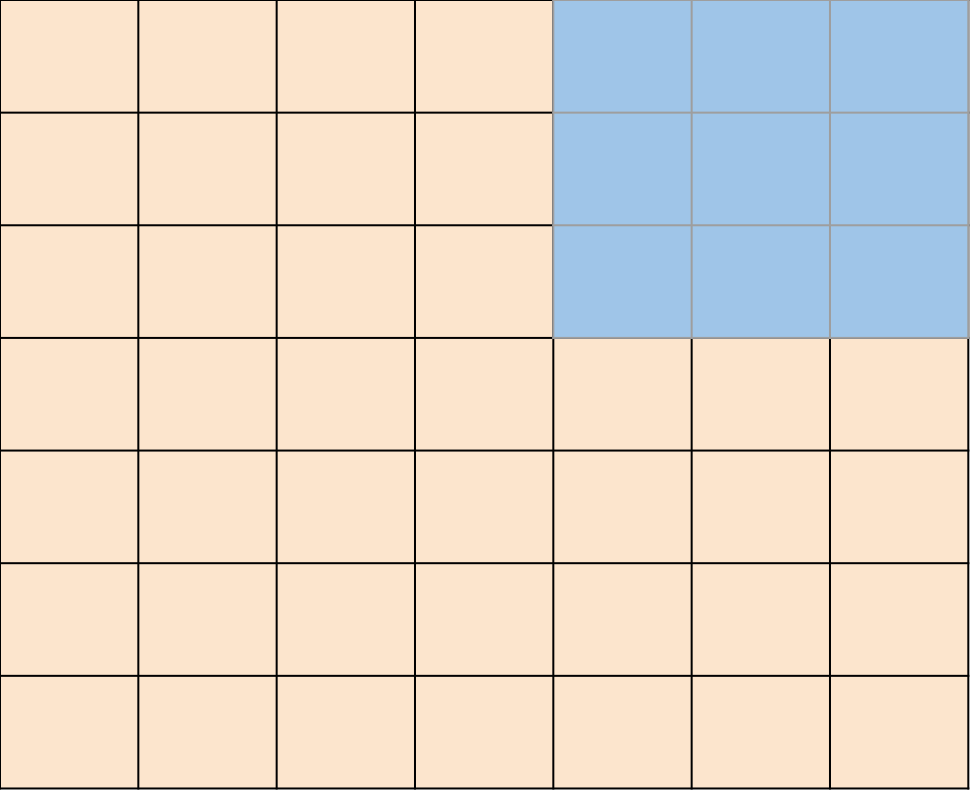
Slide 2



Slide 3



Slide 4



Slide 5

Similarly,
Spatially slide the filter along the height of the image.

Remember: At each spatial location, the dot product is added in the depth dimension

Eventually for 7x7 * 3x3 => output is 5 x 5

Hence, the output size is:

$$O = [\ (N - F + 2P) / STRIDES\ ] + 1$$

- N = DIMENSION OF INPUT IMAGE
- F = FILTER SIZE
- P = PAD SIZE
- STRIDES = NUMBER OF COL-OFFSET
- O = OUTPUT SIZE OF MAP

NOTE: NUMBER OF FILTERBANKS RESPONSES WILL BE PRODUCED

Let N = 7; F = 3

- stride 1 => (7 - 3)/1 + 1 = 5
- stride 2 => (7 - 3)/2 + 1 = 3
- stride 3 => (7 - 3)/3 + 1 = 2.33 :\

Zero Pad

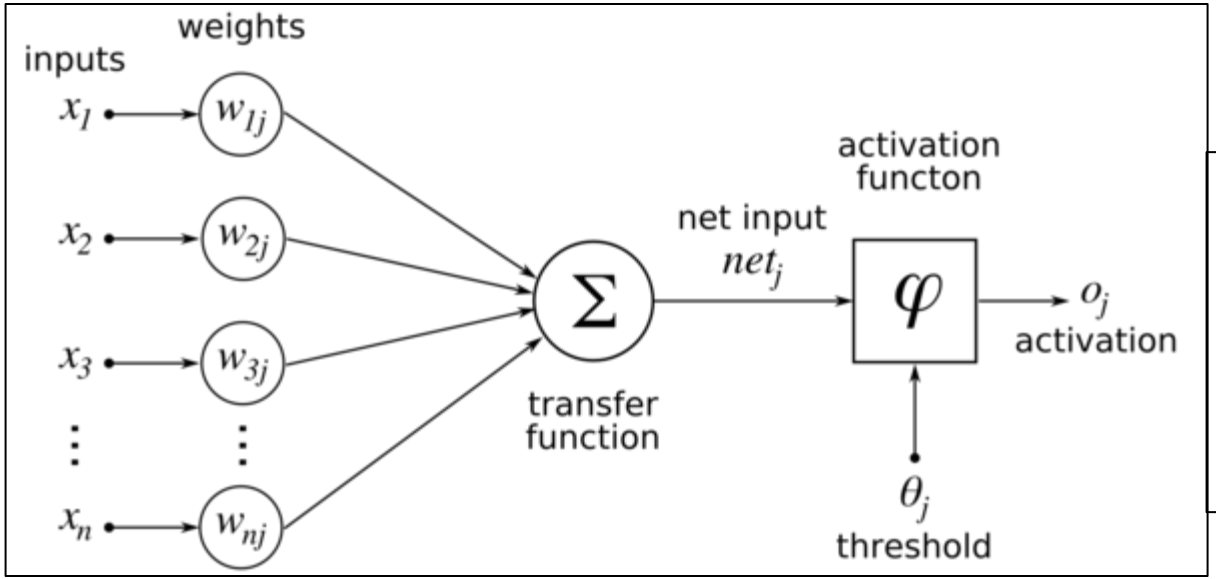
0	0	0	0	0
0				0
0				0
0				0
0	0	0	0	0

$$O = [\ (N - F + 2P) / STRIDES\ ] + 1$$

Input volume:
32x32x3 10 5x5 filters with stride 1, pad 2

Output volume size: (32 - 5 + 2*2)/1 + 1 = 32
spatially, so 32x32x10

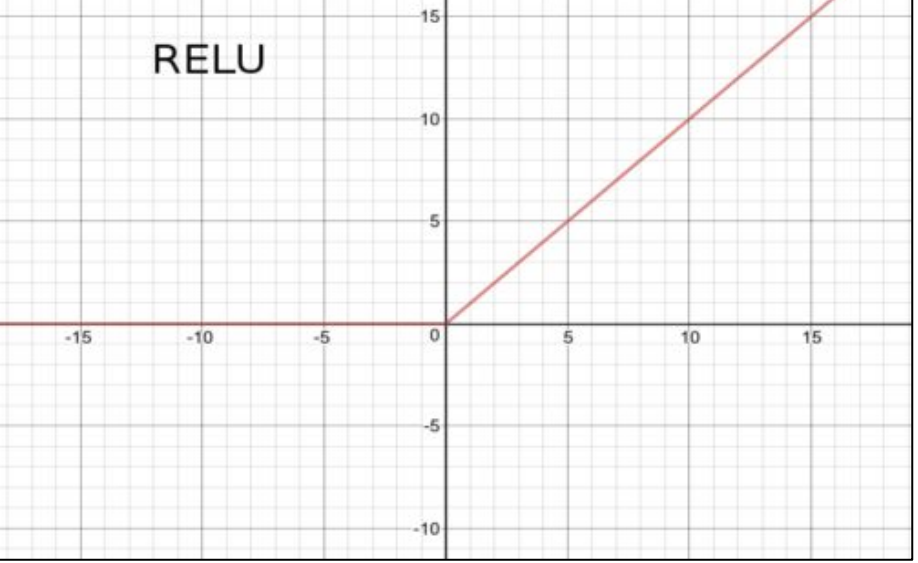
Activation : ReLU



Summation
(W.X + BIAS)

O = activation(sum)

Normal Activation in Neural Networks



ReLU : Rectified Linear Units

O = max (0 , a)

0	0	0	0	0	0	0
0	-1	1	1	8	7	0
0	12	5	-5	10	11	0
0	-11	-8	8	-4	7	0
0	5	9	-4	-10	-6	0
0	6	2	10	6	9	0
0	0	0	0	0	0	0

RELU

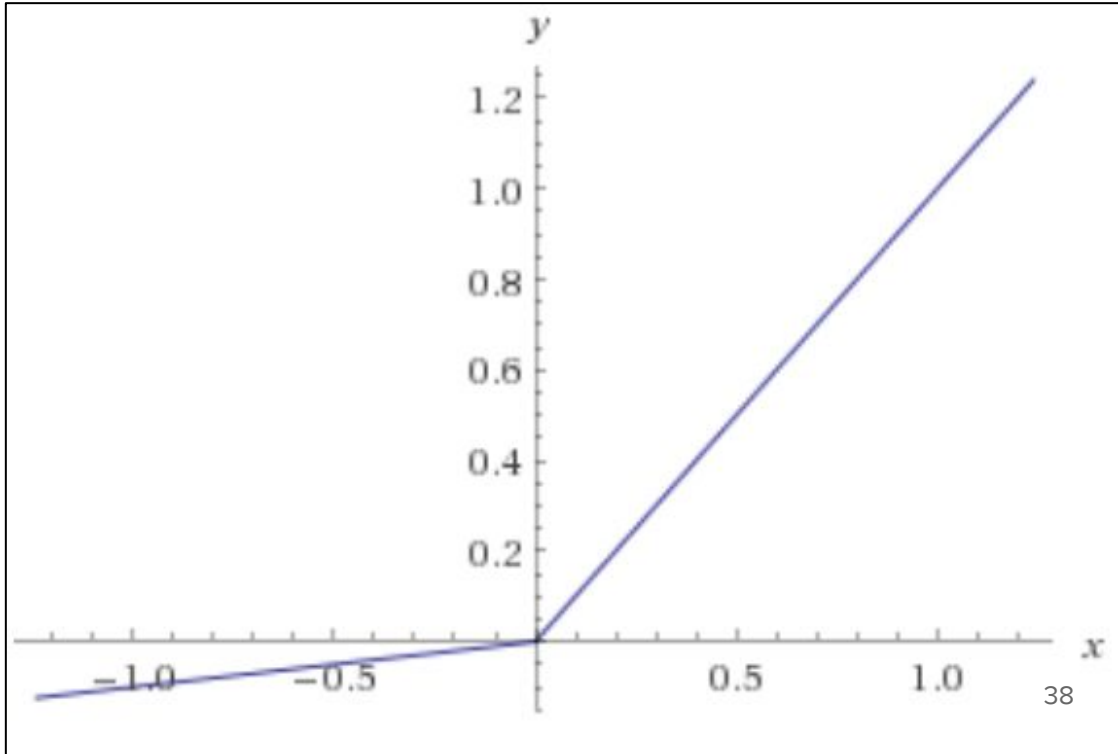
0	0	0	0	0	0	0
0	0	1	1	8	7	0
0	12	5	0	10	11	0
0	0	0	8	0	7	0
0	5	9	0	0	0	0
0	6	2	10	6	9	0
0	0	0	0	0	0	0

Other Activations

- Leaky ReLU : Solves the dead-RELU problem (update is always 0)

  O = max(0.1x, x)

- Other…
- TanH
- Sigmoid
- Similar to NN Activations



Pooling Layers : Downsampling; Makes features easily manageable

- Now,

Conv layers have been known to **DECREASE SPATIAL DIMENSION**

Input N = 32, [F = 5, S = 1, P = 0]

OUTPUT CONV #1_1 = 28

OUTPUT CONV #2_1 = 24

OUTPUT CONV #3_1 = 20

OUTPUT CONV #4_1 = 16 .....

- Hence, firstyl we
  - ZERO PADDING
- To reduce dimsionality over a certain number of conv
  - Apply pooling

Output of Pool

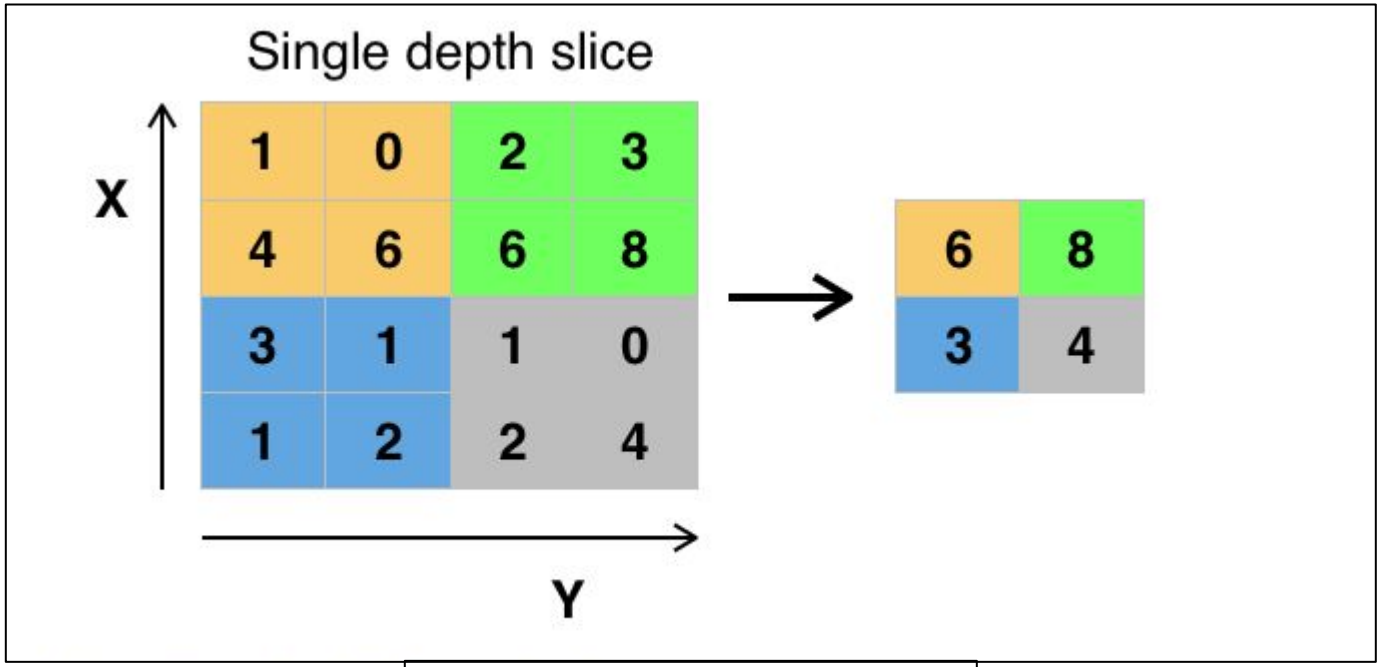
- In Pool Pad size is always 0
- Same formula as in CONV

$$O = [\, N - F / \text{stride} \,] + 1$$

- Common F = 2, S = 2

TYPES OF POOLING

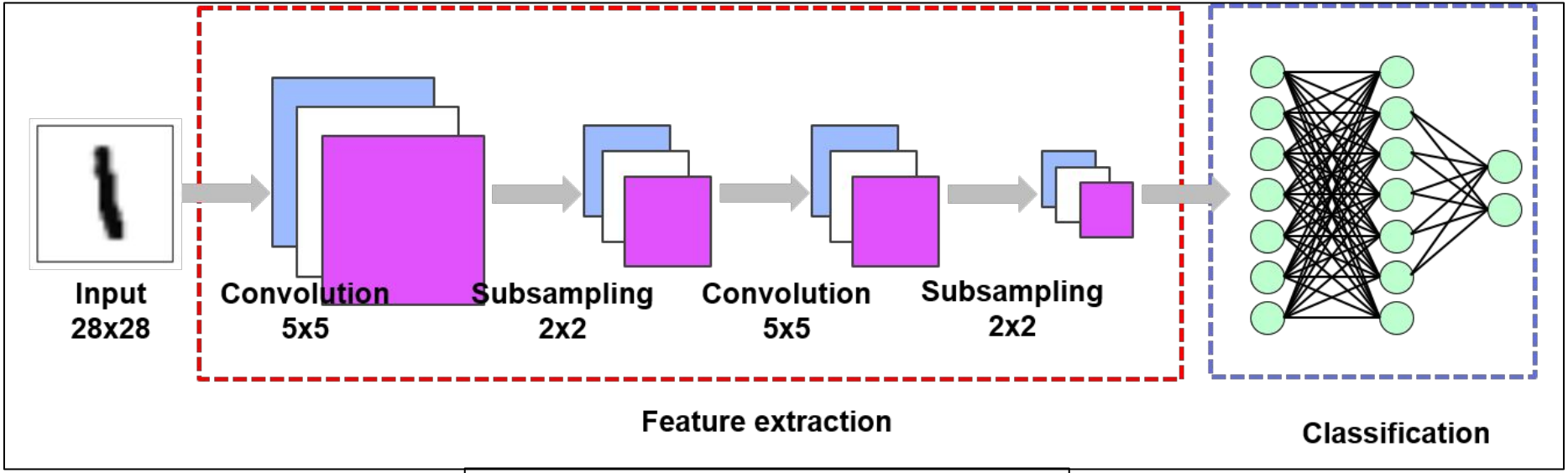
- MAX POOLING
- AVERAGE POOLING
- L2 NORM



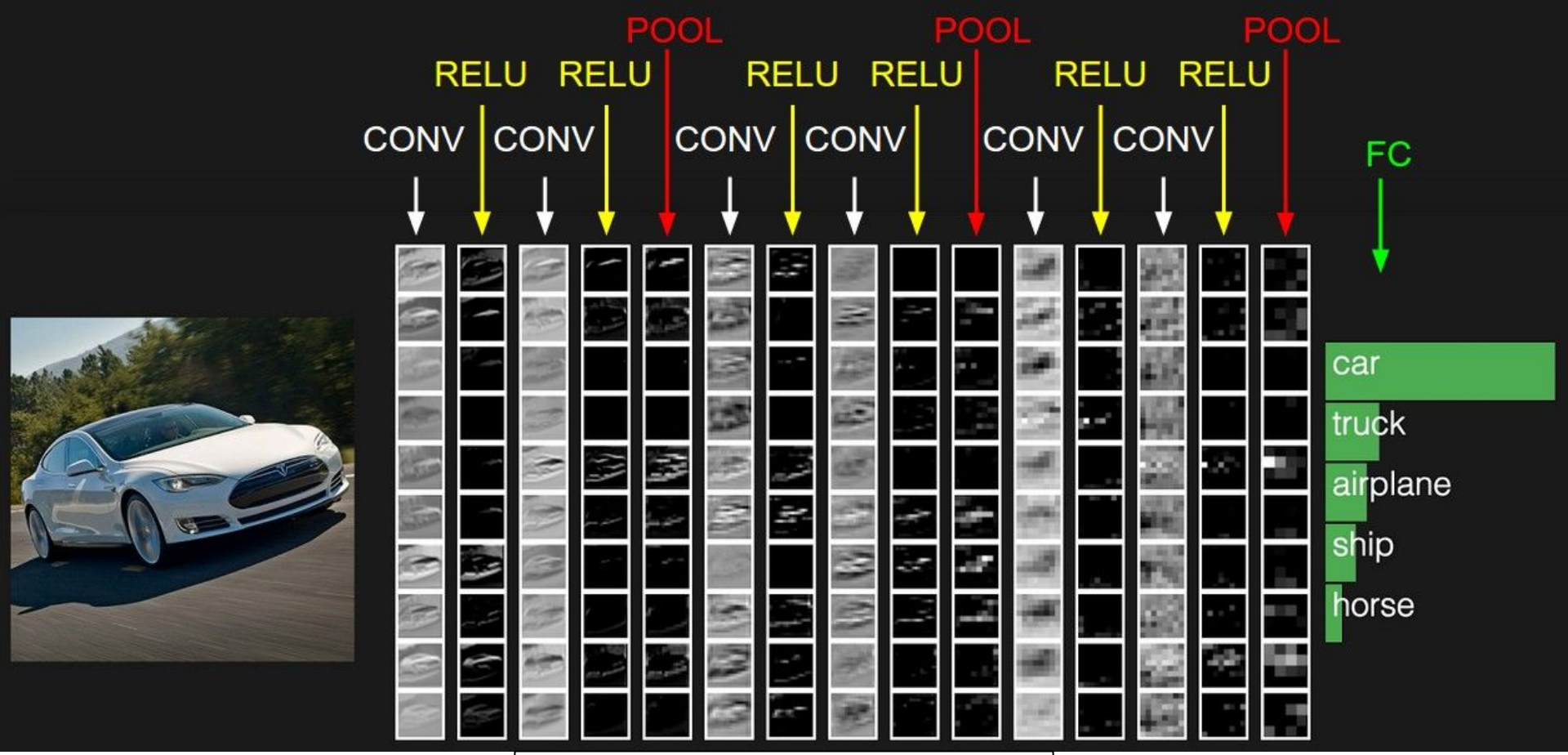
Demonstration of Max Pooling
F = 2 ; S = 2

Finally, FC Layers

Already covered #17



Convolutional Neural Network



Credits : CS231N, Stanford

Popular ConvNet Architectures

- ALEXNET
  - Input = 224 x 224 x 3 image

Details/Retrospectives: -

first use of ReLU -

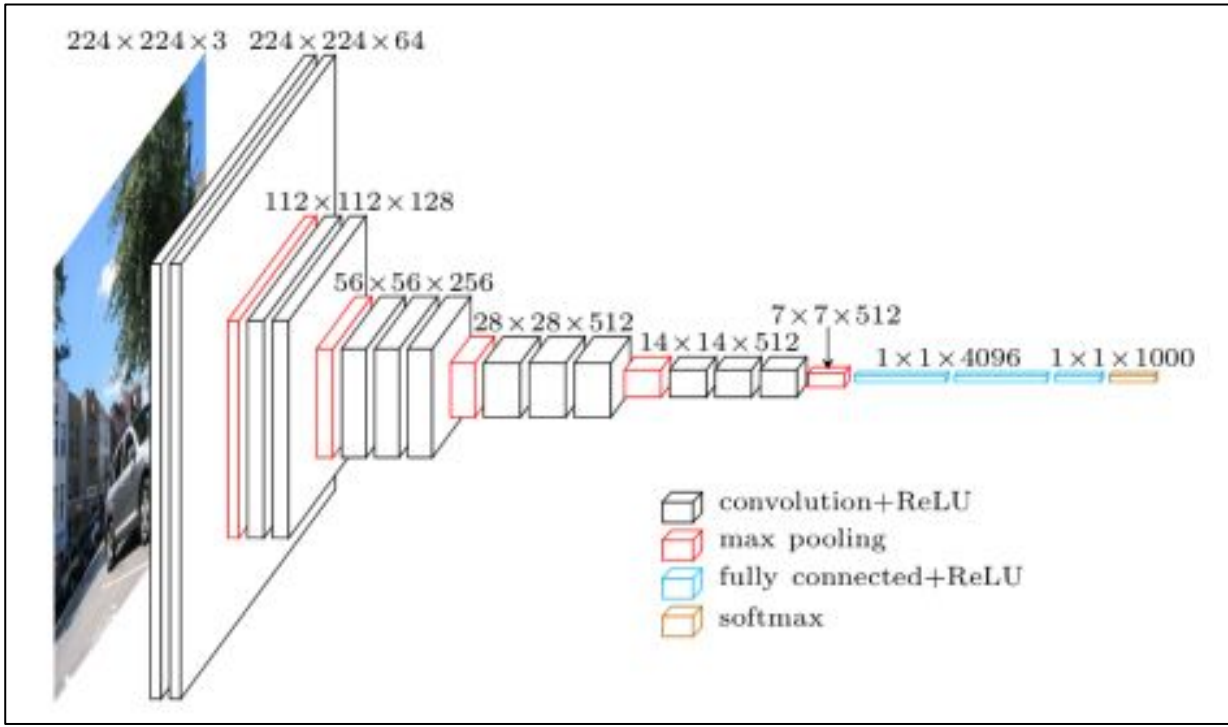
used Norm layers (not common anymore) (normalizes values in hidden layers)

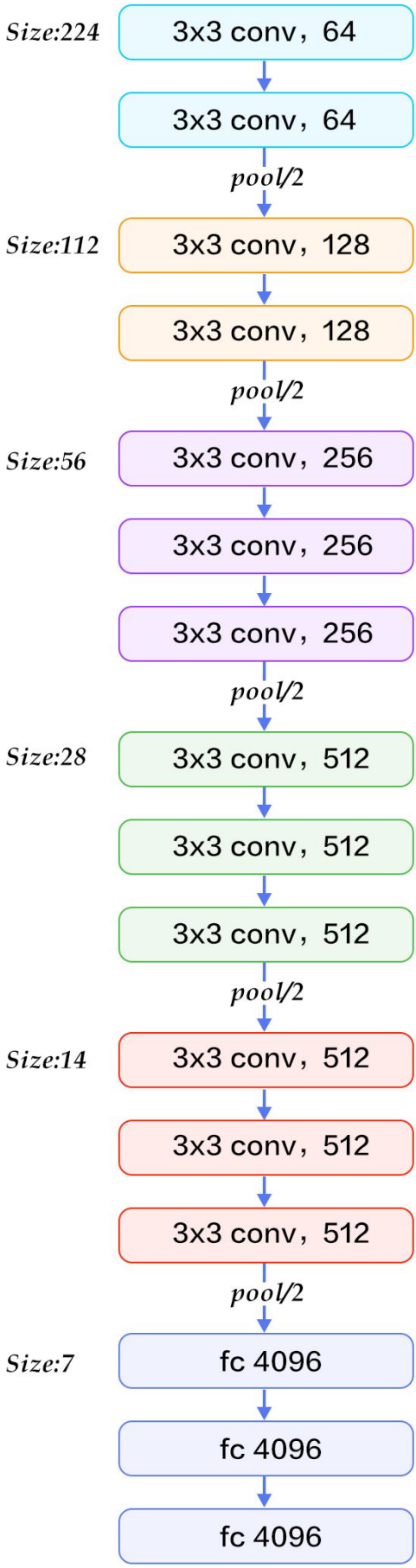
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2 [13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1 [13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1 [13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1 [6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

VGG

- Developed at Oxford
- Visual Geometry Group

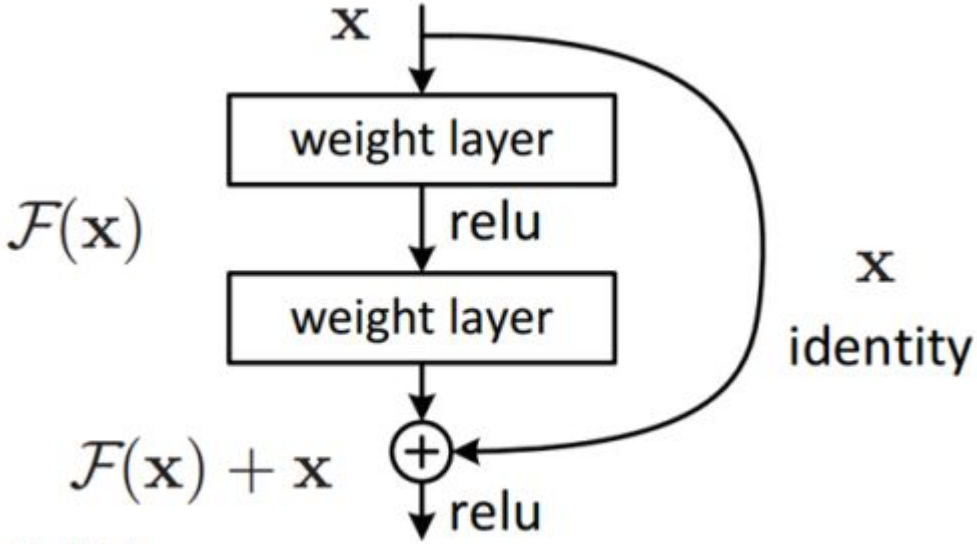
- VGG 16 :
- VGG 19 :
- Deep convolutional networks by Zisserman et.al



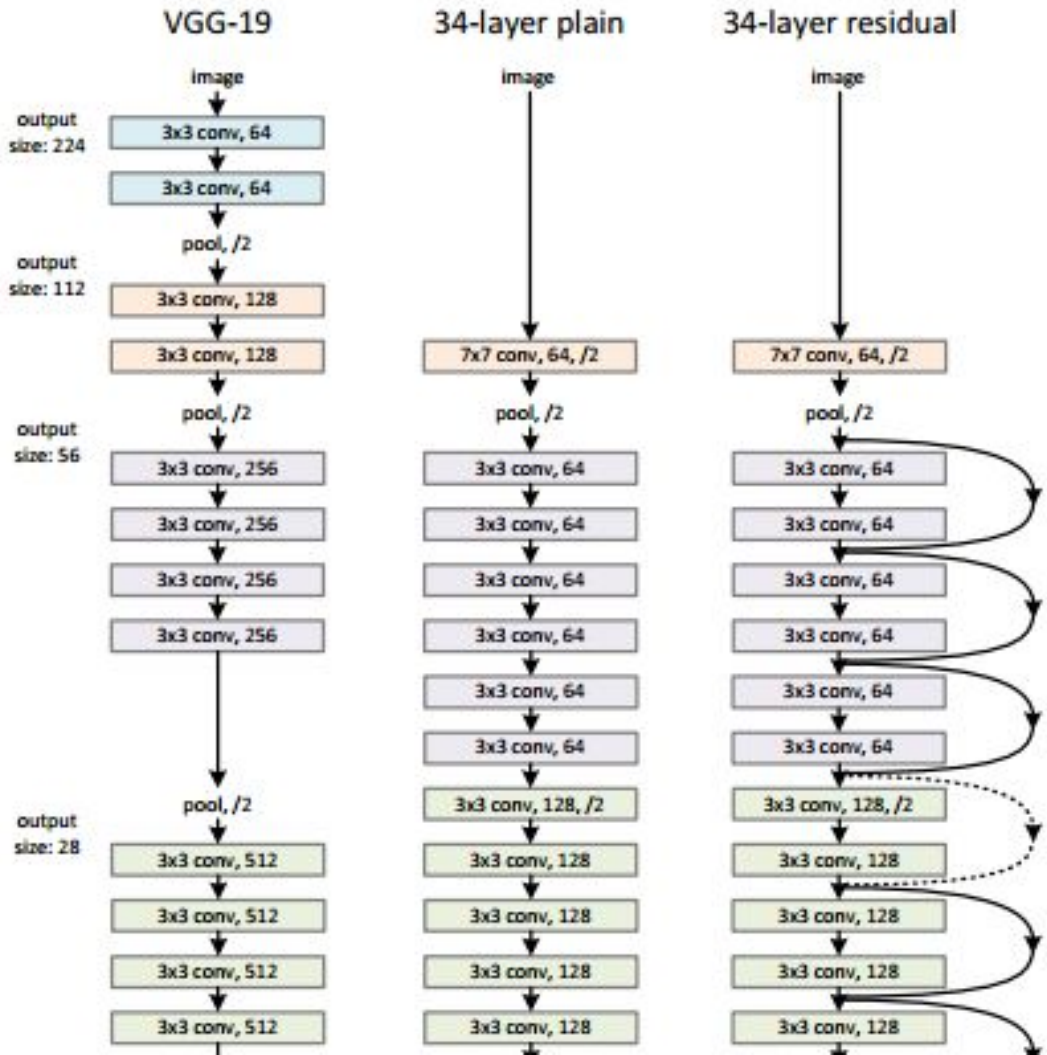

Size:224
3x3 conv，64
3x3 conv，64
pool/2
Size:112
3x3 conv，128
3x3 conv，128
pool/2
Size:56
3x3 conv，256
3x3 conv，256
3x3 conv，256
pool/2
Size:28
3x3 conv，512
3x3 conv，512
3x3 conv，512
pool/2
Size:14
3x3 conv，512
3x3 conv，512
3x3 conv，512
pool/2
Size:7
fc 4096
fc 4096
fc 4096

RESNET : Residual Networks

- Microsoft's ground breaking CNN.
- Composes of a Residual Module

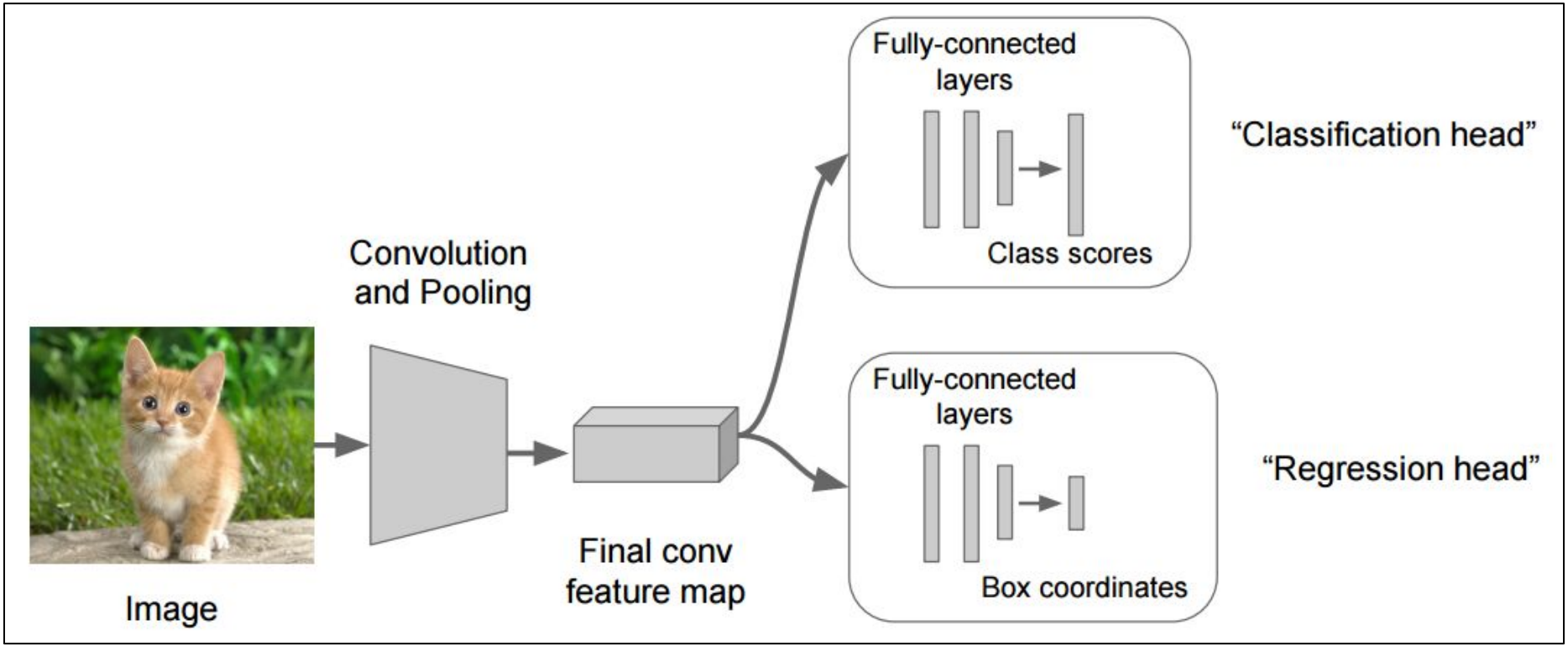


A residual block

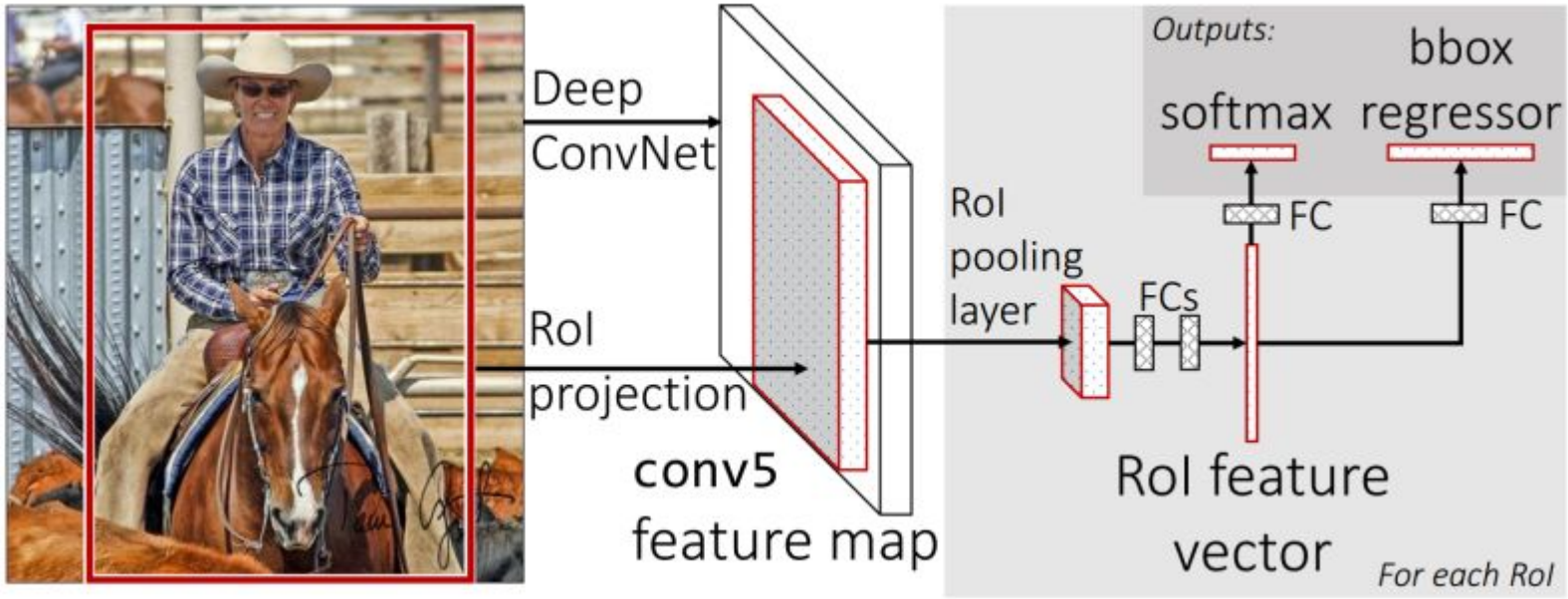


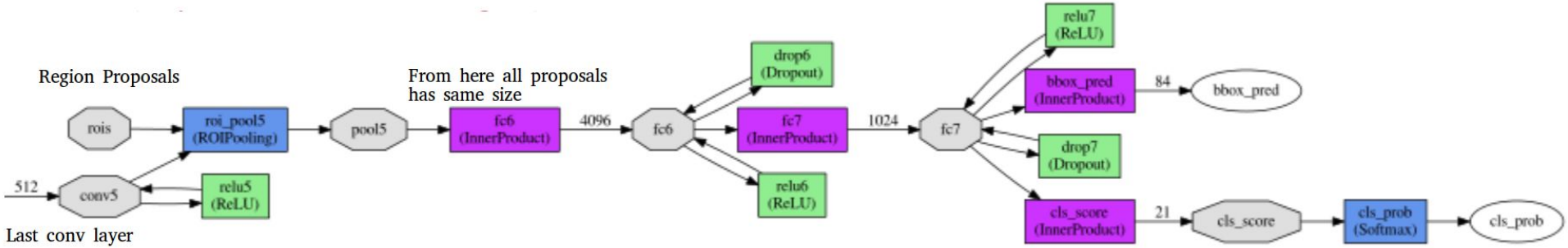
Pass on previously learned Low-level features to higher states

Localization in CNNS

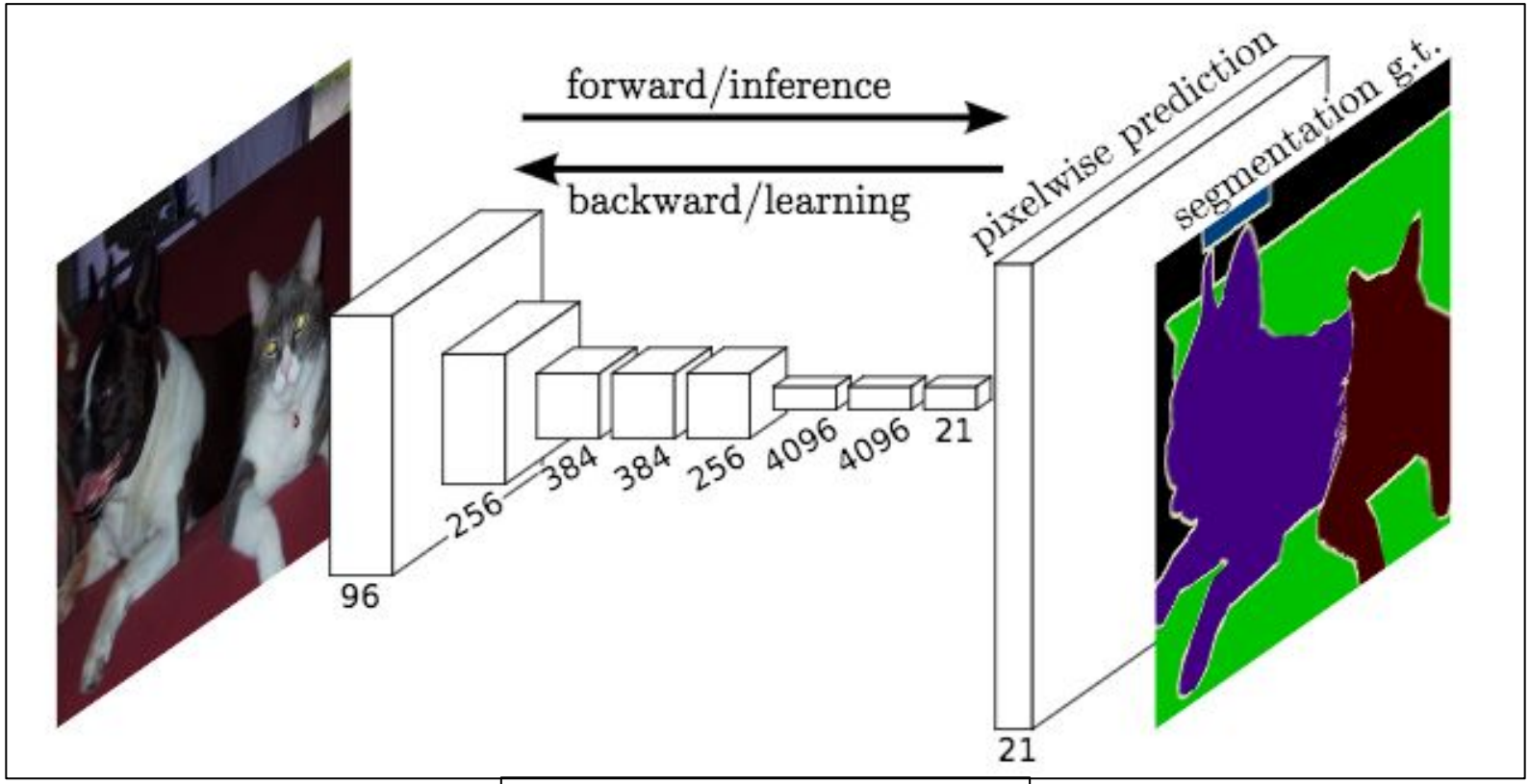


Faster RCNN


Deep
ConvNet
RoI
projection
conv5
feature map
RoI
pooling
layer
FCs
Outputs:
softmax
bbox
regressor
FC
FC
RoI feature
vector
For each RoI


Region Proposals
From here all proposals
has same size
rois
roi_pool5
(ROIPooling)
pool5
fc6
(InnerProduct)
4096
fc6
drop6
(Dropout)
fc7
(InnerProduct)
1024
fc7
relu7
(ReLU)
bbox_pred
(InnerProduct)
84
bbox_pred
drop7
(Dropout)
cls_score
(InnerProduct)
21
cls_score
cls_prob
(Softmax)
cls_prob
512
conv5
relu5
(ReLU)
relu6
(ReLU)
Last conv layer

Fully Convoluted Networks - Semantic Segmentation

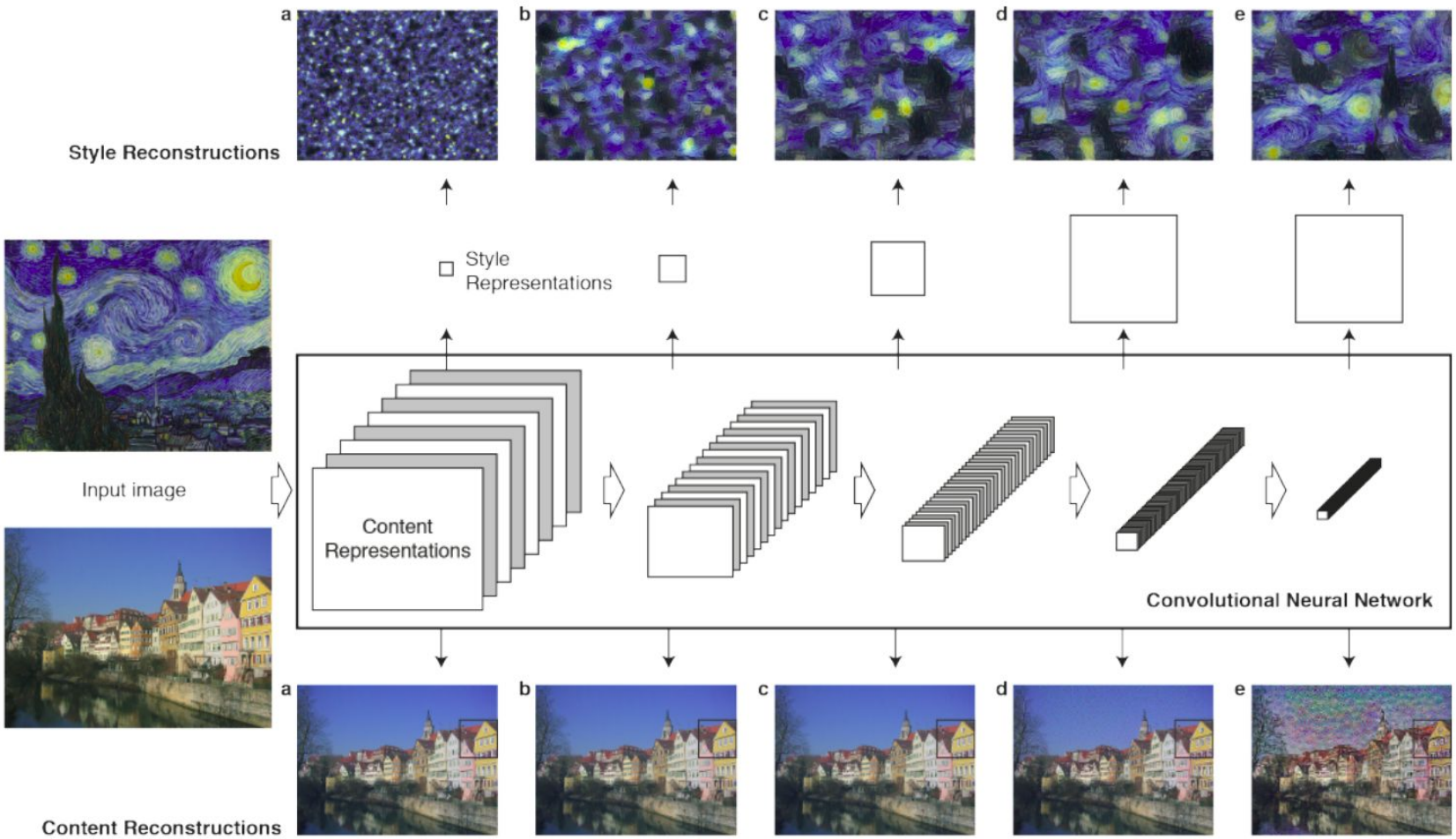


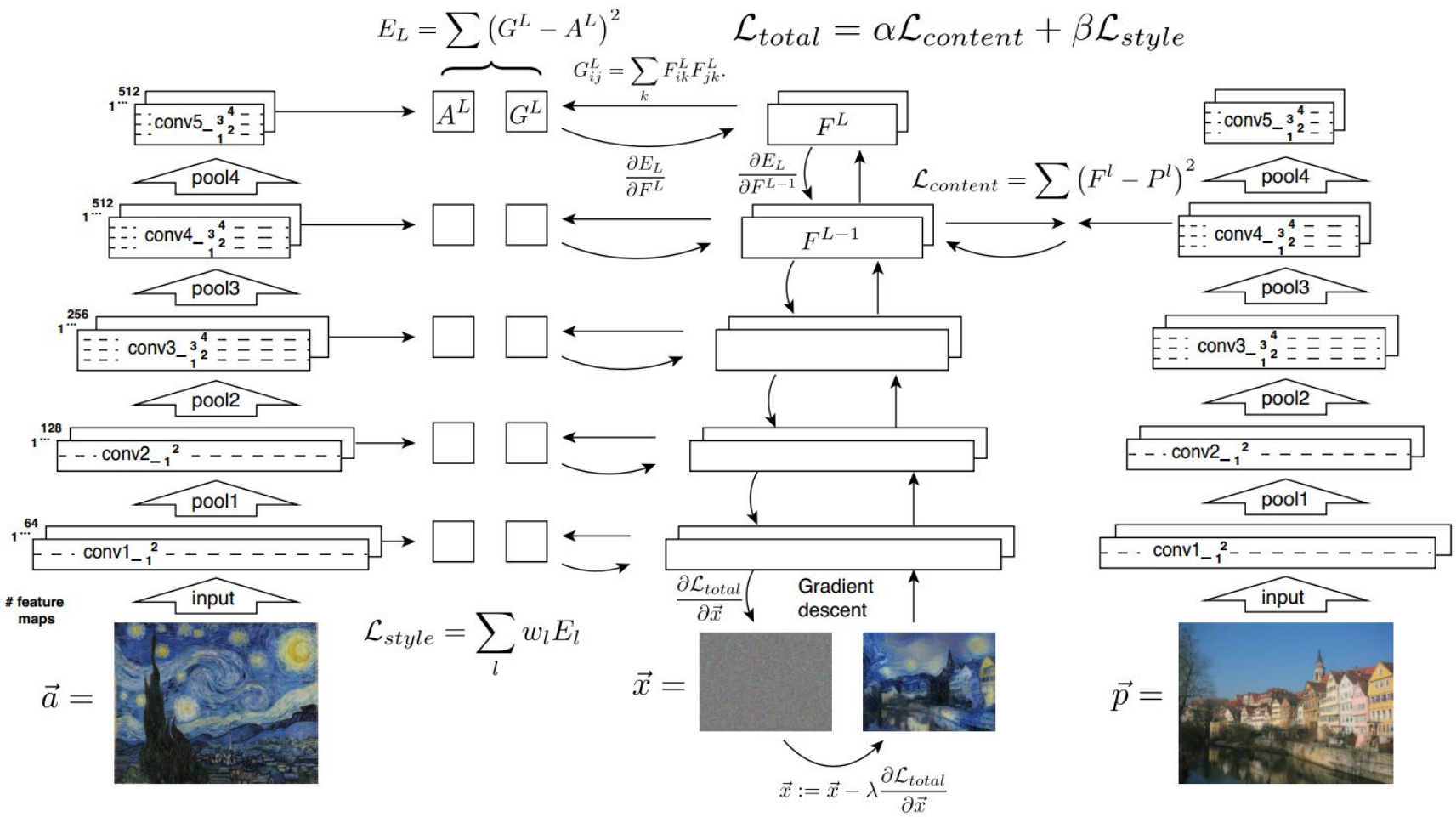
Evan Shelhamer et.al. UCB

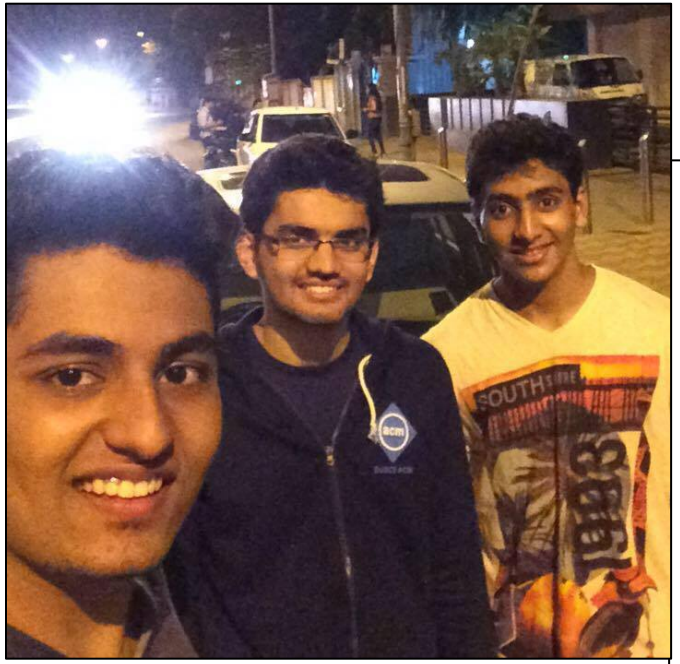
Style Transfer : Leon Gatys et. al.

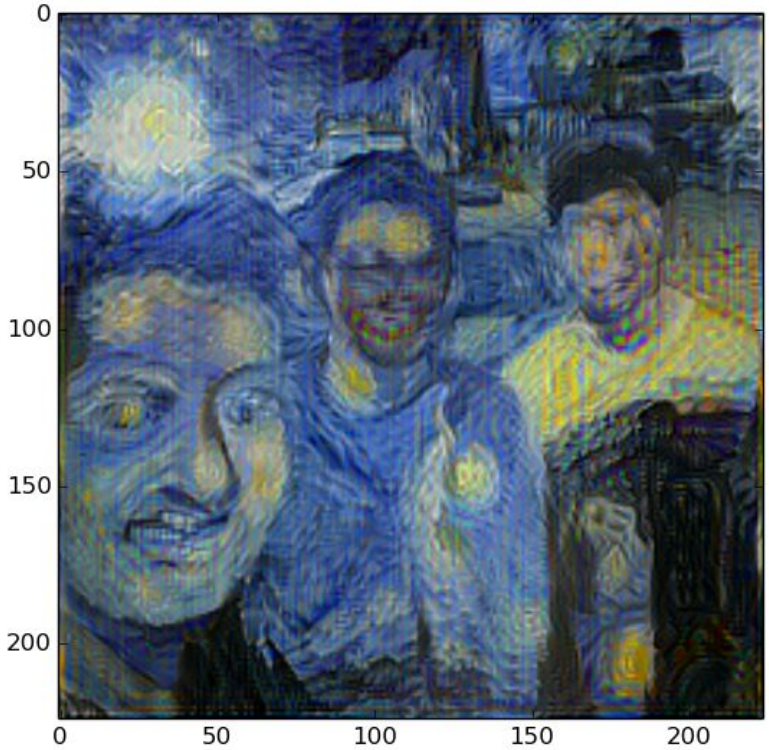
- Convnets represent feature and content Hierarchy
- Minimize style and content loss

$$\mathcal{L}_{\text{total}}(\vec{p}, \vec{a}, \vec{x}) = \alpha \mathcal{L}_{\text{content}}(\vec{p}, \vec{x}) + \beta \mathcal{L}_{\text{style}}(\vec{a}, \vec{x})$$


a
b
c
d
e
Style Reconstructions
Style
Representations
Input image
Content
Representations
Convolutional Neural Network
Content Reconstructions


$E_L = \sum (G^L - A^L)^2$
$\mathcal{L}_{total} = \alpha \mathcal{L}_{content} + \beta \mathcal{L}_{style}$
$G^L_{ij} = \sum_k F^L_{ik} F^L_{jk}.$
A^L
G^L
F^L
F^{L-1}
$\frac{\partial E_L}{\partial F^L}$
$\frac{\partial E_L}{\partial F^{L-1}}$
$\mathcal{L}_{content} = \sum (F^l - P^l)^2$
conv5_ 3 4 2 1
pool4
conv4_ 3 4 2 1
pool3
conv3_ 3 4 2 1
pool2
conv2_ 1 2
pool1
conv1_ 1 2
input
512
512
256
128
64
feature maps
$\frac{\partial \mathcal{L}_{total}}{\partial \vec{x}}$
Gradient descent
$\mathcal{L}_{style} = \sum_l w_l E_l$
$\vec{a} =$
$\vec{x} =$
$\vec{p} =$
$\vec{x} := \vec{x} - \lambda \frac{\partial \mathcal{L}_{total}}{\partial \vec{x}}$





1000 iterations
Style Transfer
Implemented in Caffe

Tips

```
layer {
    name: "conv1"
    type: "Convolution"
    bottom: "data"
    top: "conv

    ....
    ....
    ....

    convolutio
        num_output: 96
        kernel_size: 11
        stride: 4
        ...
        ...
        ...
    }
}
```

Caffe

TensorFlow

Tf.nn.conv2d (parameters ….)

K

keras.layers.Conv2D( )